

CRP Programming Instruction

This system supports several types of robot, and make a part instruction about the six-joint robot. Please refer to this instruction and debug manual for other type of robots.

Please make sure the operator of this product
accepted the instruction related

Safety Attentions

Please read carefully and understand totally of these instructions and other related documents BEFORE using this system. Please start using the system ONLY AFTER mastering all equipment knowledge, safety knowledge, and notices.

Safety attentions in this instruction are graded into four categories- DANGER, CAUTION, MUST, and FORBID



DANGER

indicating that death or severe injury will result if wrong operation is taken.



CAUTION

indicating that medium or slight injury or equipment breakdown will result if wrong operation is taken.



MUST

instructions that must be obey.



FORBID

operations that mustn't be taken.

It should be noticed that things labeled with CAUTION may also result into severe consequences in different situation. So it's very important to pay attention to every CAUTION instructions, and stay strictly with them.

Some instructions even may not be labeled with DANGER or CAUTION, but users should also obey them.

! DANGER

★Before operating the robot, press the “emergency stop” button on the teach box and make sure main power supply of servo is off and motor is in “off-power and brake” status. After cutting off the servo power, servo power button on the teach box is red.

Under an emergency, personal injury or equipment damage may result if robot cannot be stopped in time.



★When emergency is removed and servo power need connecting, please remove and deal properly with the incidence which causes the emergency, and then connect the servo power.

Personal injury may result due to the unwanted robot movement from wrong operation.

rotate the button according the white arrow to remove the emergency stop.



★When teaching within the robot movement region, please obey the following principle: keep observe the robot from the front

follow the operation manual strictly

consider the backup plan if robot move suddenly toward yourself

set up the shelter just in case

Personal injury may result due to the unwanted robot movement from wrong operation.

★When following processes are operated, make sure nobody is in the robot movement region and operator is in the safe position: powering the robot control electric tank

controlling the robot using teaching programmer

test running of robot

reoccurrence automatically

Personal injury may result when entering the robot movement region or contacting the robot.

Press the emergency stop button immediately when exception occurs.



★Following items need confirming before operating robot:

Operator is well-trained with robot-operation;

Operator has adequate knowledge of the movement feature of robot;

Operator is fully aware of the danger of robot;

Operator hasn't drink alcohol;

Operator hasn't take drugs that affect the nerve system and make reaction slow.

★Following items need checking before robot teaching process. If any abnormal is found, fix it or take other necessary measurement immediately.

Movement of robot is normal or not;

Home point adjustment is correct or not;

Auxiliary equipment connected to robot is normal or not.

★Please put the teach equipment to original position after usage, and place it firmly.

Please don't put the teach programmer on robots, claps or ground. Otherwise, personal injury or equipment damage may result when teach programmer has collision with robot or claps.

Please be cautious with the incidental falling of teach programmer, which may lead to wrong movement of robot, personal injury and equipment damage.



Safe operation protocols

★All the operators of robot systems should participate in system training, study safety protection measurements and understand robot functionality.

★Make sure no abnormal or dangerous situation of robot and auxiliary equipment occurs, before starting operating robot.

★ Unplug the power or press emergency stop button before entering the operation region, even robots aren't running.

★ When programming within the movement region of robot, corresponding guard should be set to make sure that robot can be stopped under emergency circumstances. Please don't wear gloves when teach and inch the robot. Please take the slow mode when inching the robot, in case emergency occurs and robot can be stopped efficiently.

★Operators must be familiar with the emergency stop buttons on the robot controller and periphery controller, to the degree that these buttons can be pressed exactly under emergency.

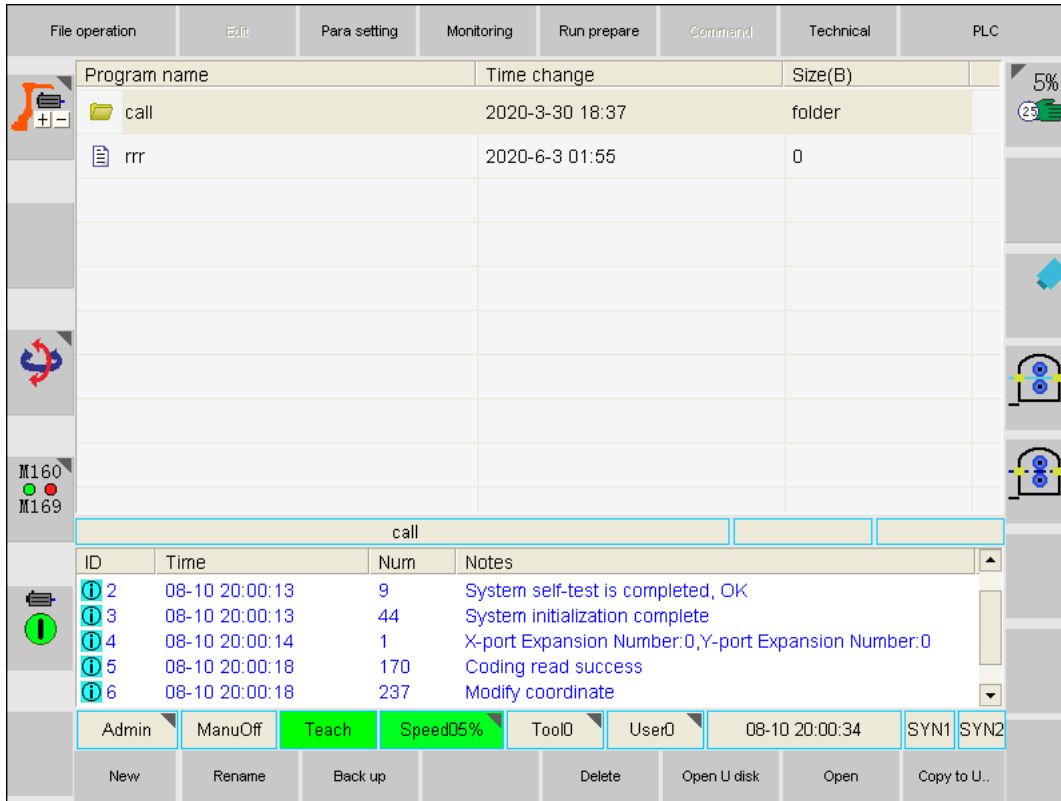
★Never think that static status of robot means the finish of program. Static robot is likely to receive the input signal to get moving.

Catalog

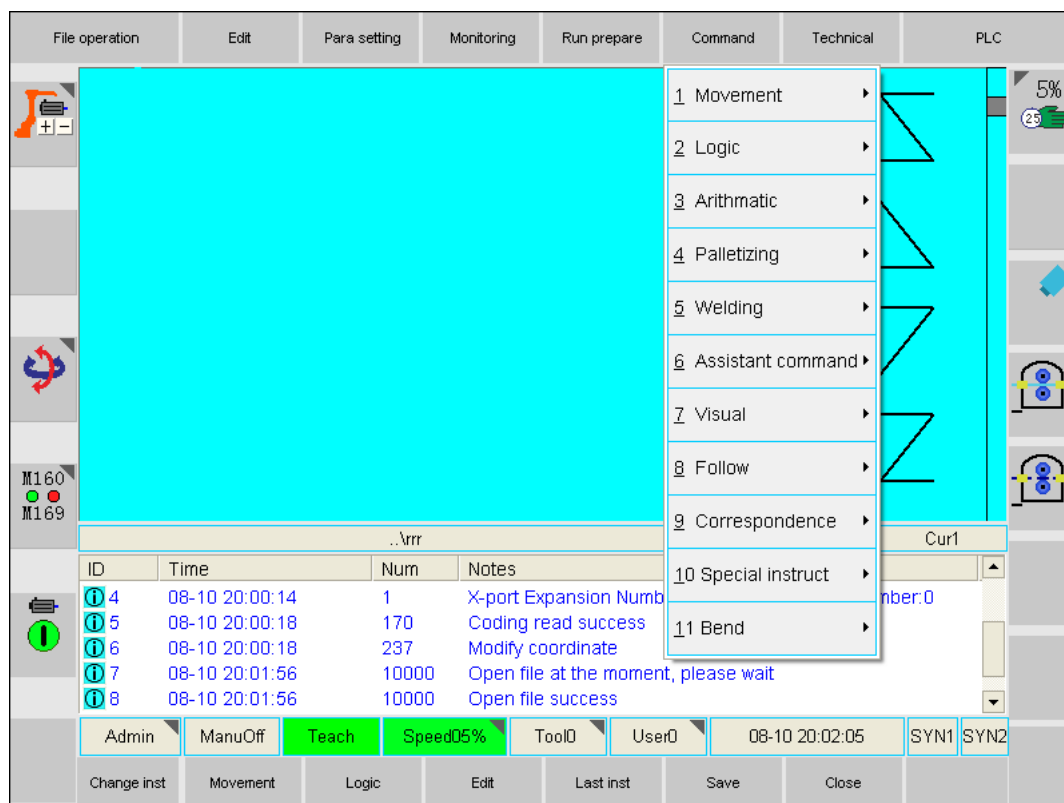
1 Invoke of program instruction	7
2 Description of Program Commands	9
3 Structure of Program Command:	9
4 Description of Additional Options:	10
5 List of Program Commands:	14
6 Description of Program Commands:	17
6.1 Movement instrument	17
6.2 Logic Instructions	22
6.3 Calculation instructions	26
6.4 Pallet instruction	29
6.5 Welding instructions	30
6.6 Painting Instruction (Backup Function)	31
6.7 Pouring Instruction (Backup Function)	31
6.8 Auxiliary Function	31
6.9 Vision Instruction	43
6.10 Track instructions	45
6.11 Communication instructions	46
6.12 Special Instructions	47
7 Teach program	49
7.1 Welding teach program	50
7.2 Lifting program example	54
8 Program edit functions	60
8.1 Edit in Program list	60
8.2 Edit in program edit interface	63
9 Program test-run	71
9.1 Preparation:	71
9.2 Steps for program test-run:	71

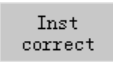
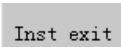
1 Invoke of program instruction

1. Turn the mode security switch to teach mode.
2. Switch to the program list interface, create or open a program.



3. Click the **Command** button in the status control bar, or click the **Movement** > **Logic** button in the functional button area to invoke the frequently used instructions. When using functional area buttons, repeat clicking the button will result into the different instructions under the menu. Click **Last inst** button to invoke the instructions used last time.



4. After the input of correct instruction parameters, click <  > button to input the instruction; click the <  > button to cancel the instructions input.

These two operations should be carried out while pressing the security switch.

2 Description of Program Commands

1 . <> contains the selected or input content. [Blank] contains nothing, meaning don't use.

3 . Program instruction line should select or input the following types of content: instructions, judgement symbol, data, status, number.

Instructions: instructions need invoking, e.g. MOVJ, JUMP, GP, LI, etc.

Judgement symbol: Used to judge the condition relationship, e.g. ==, >, <,>=, <=.

Assignment: Set the certain port into the certain condition, e.g. Y0=ON, M24=OFF.

Data: Data can be divided into digital, integer, minus, and characters according to the different instructions.

Status: (ON=1, OFF=0). The status can be used as the number 0, 1.

Number: Number should be integer larger than 0 (0 included) according to the different instructions. The number range may also different.

4 . The additions may differ attached to the different instructions. Please pay attention to the additions.

5 . Positions with downwards arrow symbol  should be inputted by <up/down arrows>selecting rather than directly input. Otherwise errors may occur.

6 . Please select the instructions according to the reality where many instructions are available in additional instructions position.

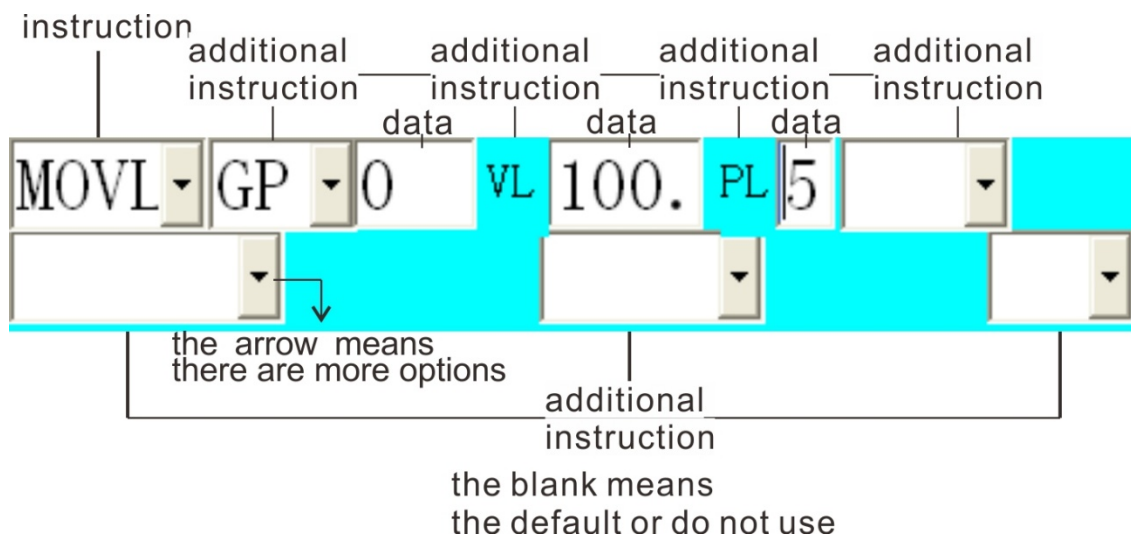
7 . 【Shortcut: 8 5 1】 Invoke the instructions using shortcut. After showing the program instruction list, instructions can be invoked by using number button, arrow button, and enter button. Examples are as following:

【Shortcut: 8 5 1】 Press 8,5,1to invoke SWITCH

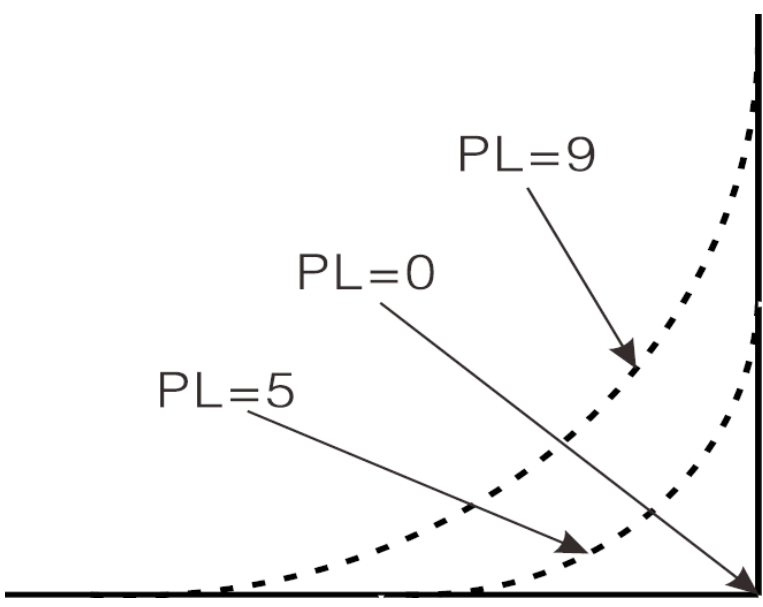
【Shortcut: 2 1 E】 Press 2,1, enter button to invoke RET

【Shortcut: 8 ↑ ↑ E】 Press8, upward arrow, upward arrow, enter button to invoke FORBIDAXISEND.

3 Structure of Program Command:



4 Description of Additional Options:

Additional option formats and definition	Descriptions
VJ=<%>: Joint movement speed	Units: 1%, Max: 100%. Actual speed = highest speed of the axis in parameters* VJ * automatic operation multiplier
VL=<straight movement speed>: straight movement speed	Units: MM/S, Max: Highest straight movement speed in parameters. Actual speed = VL*automatic operation multiplier
PL=<Smoothness>: Smoothness	Range 0-9. Basically, it's the radian of transition, which determines if the transition is performed in rectangular or circle way. Suppose two straight lines need connecting, parameters should be set to determine the radian of the transition line. The below figure is the scheme of value selection. 
UNITL <condition>: Condition judgement	Parameters can be used to judge: X<variable #>, M<variable #>, T<variable #>, C<variable #>; The condition of user judgment is effective (ON=1), ineffective (OFF=0). E.g. UNTIL X# (0) ==ON to judge X0 port is effective; UNTIL M# (3) ==OFF to judge the M3 auxiliary relay is ineffective.
X<Variable >: General input X port	General input X port: this port is related to hardware physical port. Status variable (ON=1, X<variable #>: general input OFF=0), range: 0-111. Port status can be monitored in the "Monitor"- "I/O port"- "general input port monitor". variable # can be referred to "Hardware operation manual"
M<variable #>: Internal auxiliary 0-800.	Internal auxiliary M relay. Status variable (ON=1, OFF=0), range: 0-800 The function definition, variable # of the auxiliary relay can be referred to "PLC manuals". The status of relay can be monitored in the "Monitor"- "PLC"- "M".

GI<variable #>: global I variable	range: 0-99, values should be integer, with positive or negative symbol. So when Program first invokes the same variable number, the value of the variable is same. When Program secondly invokes the variable number, the value of the variable will be covered. This variable can be monitored in “operation preparation”-“variables”-“global I variables” list.
LI<variable #>: local I variable	range: 0-1000, values should be integer, with positive or negative symbol. When different programs invoke the same variable number, the correspondent values are not same, and will not interface and independent. This variable can only be shown in the variable list when the program is open.
GD<variable #>: Global D variable	range: 0-99, values should be float, with three decimal places (.000) and positive or negative symbol. So when Program first invokes the same variable number, the value of the variable is same. When Program secondly invokes the variable number, the value of the variable will be covered. This variable can be monitored in “operation preparation”-“variables”-“global D variables” list.
LD< variable #>: local D variable	range: 0-1000, float variable, values should be float, with three decimal places (.000) and positive or negative symbol. When different programs invoke the same variable number, the correspondent values are not same, and will not interface and independent. This variable can only be shown in the variable list when the program is open.
GP< variable #><data number>: Global P variable	Global P variable, combination of multi-data, used to record the joint gesture, coordination, etc., of lathe. Variable range: 0-999, number range: 0-11. Data number refers to the certain data in the GP variables combination. Data number is defined as: 0: operate the whole data combination; 1: operate the x axis data; 2: operate the y axis data; 3: operate the z axis data; 4: operate the J1 axis data; 5: operate the J2 axis data; 6: operate the J3 axis data; 7: operate the J4 axis data; 8: operate the J5 axis data; 9: operate the J6 axis data; 10: operate the J7 axis data; 11: operate the J8 axis data; So when Program first invokes the same variable number, the value of the variable is same. When Program secondly invokes the variable number, the value of the variable will be covered. So when using the previously used variable number, please pay attention in case of wrong movement or danger.

	This variable can be monitored in “operation preparation”-“variables”-“global P variables” list.
LP< variable #><data number>: Local P variable	Local P variable, combination of multi-data, used to record the joint gesture, coordination, etc., of lathe. Variable range: 0-1000, number range: 0-11. Data number refers to the certain data in the LP variables combination. Data number is defined as: 0: operate the whole data combination; 1: operate the x axis data ; 2: operate the y axis data ; 3: operate the z axis data : 4: operate the J1 axis data ; 5: operate the J2 axis data ; 6: operate the J3 axis data ; 7: operate the J4 axis data ; 8: operate the J5 axis data ; 9: operate the J6 axis data ; 10: operate the J7 axis data ; 11: operate the J8 axis data ; When different programs invoke the same variable number, the correspondent values are not same, and will not interface and independent. For example, when one program invokes variable L90, and another program invokes variable L90, these two L90 variables are independent and will not interface. This variable can only be shown in the variable list when the program is open.
Y<variables>: general output port	General output port, which is related with hardware physical port. Status variable (ON=1, OFF=0), range 0-79. This variable can be monitored in “Monitor”-“IO port”-“general output monitor”. Variable number is referred to “Hardware manual”.
T<variable #>: Timer	T<variable #> Internal timer number, status variable (ON=1, OFF=0), range 0-59. The status of this variable is ON or OFF. Status of timer can be monitored in “Monitor”-“PLC”-“monitor of timer”.
TC<variable #>: value in the timer	T<variable #>internal timer data, number variable. This data can be monitored in “Monitor”-“PLC”-“monitor of timer”.
C< variable #>: counter	C<variable #>internal counter number, status variable (ON=1, OFF=0), range 0-19. This counter status can be monitored in “Monitor”-“PLC”-“monitor of counter”.
CC< variable #>: value in the counter	C<variable number>internal counter data, number variable, range 0-19. This data can be monitored in “Monitor”-“PLC”-“monitor of counter”.
GS< variable #>: global character string variable	GS character string variable, range 0-99. This variable is mainly used to read the barcode and to edit data from external I/O port. This variable should be used with READ DATA TO or READ IO BCD instructions. E.g.: READDATATO GS# (0): read barcode to variable GS0 PAUSE IF GS# (0) =123456: pause whenGS0=123456

	
D< variable >: user-defined number variable	User-defined number variable, which is the specific data user can input. This variable can use +/- symbol, and can be a decimal.
==: judge equal	Judge if two variables are equal or not. Both number variable and status variable is available. E.g.: X0==ON; LI0=LI2; etc.
>: judge greater than	Judge if the former variable is greater than the later one. Number variable is used here. E.g.: LD0 > LI2; CC0>LD1; etc.
<: judge less than	Judge if the former variable is less than the later one. Number variable is used here. E.g.: LD0<LI2; CC0<LD1; etc.
>=: judge equal or greater than	Judge if the former variable is equal to or greater than the later one. Number variable is used here. E.g.: LD0 > =LI2; CC0>=LD1; etc.
<=: judge equal or less than	Judge if the former variable is equal to or less than the later one. Number variable is used here. E.g.: LD0<=LI2; CC0<=LD1; etc.
=: Setting symbol	Set the former variable into the certain status, e.g.: Y0=ON, M24=OFF, etc.

5 List of Program Commands:

Type	Commands	Description
1 Movement	1 MOVJ	
	2 MOVL	
	3 MOVC	
2 Logic	1 DOUT	
	2 AOUT	
	3 WAIT	
	4 TIME	
	5 PAUSE	
	6 JUMP	
	7 CALL	
	8 ;	
	9 *	
	10 RET	
	11 TDOUT	
3 Calculation	1 ADD	
	2 SUB	
	3 MUL	
	4 DIV	
	5 INC	
	6 DEC	
	7 SET	
4 Lifting	1 PALLET	
5 Welding	1 ARC START	
	2 ARC END	
	3 WEAVE	
	4 WEAVE END	
6 Painting	1 GUN ON	
	2 GUN OFF	
	3 GAS ON	
	4 GAS OFF	
7 Pouring		

8 Auxiliary Instructions	1 SPEED		
	2 IF	1 IF	
		2 ELSE IF	
		3 ELSE	
		4 END IF	
	3 FOR		
	4 WHILE	1 WHILE	
		2 END WHILE	
	5 SWITCH	1 SWITCH	
		2 CASE	
		4 DEFAULT	
		5 BREAK	
		6 END SWITCH	
	6 PULSE TARANSFORM		
	7 PULSE TARANSFORM END		
	8 IMPUTED		
	9 ZERO		
	10 MOTION FINISH		
	11 CHANGE USE		
	12 FORBID AXIS		
	13 FORBID AXIS END		
	14 COUNT COORD		
	15 CHANGE USE		
10 Vision	1 RUN VISION		
	2 GET VISON DATA		
	3 CLEAR VISON DATA		
	4 TRIGGER VISON		
	5 CHANGE VISON		
11 Track	1 TARCK START		
	2 TARCK END		
	3 GET TRACK DATA		
	4 CLEAR TARCK DATA		
	5 RUN IO CUTIN		
12 Communication	1 READ IO BCD		
	2 READ DATA TO		

13 Special instructions	1 RESET	
	2 CHANG EPOINT	

6 Description of Program Commands:

6.1 Movement instrument

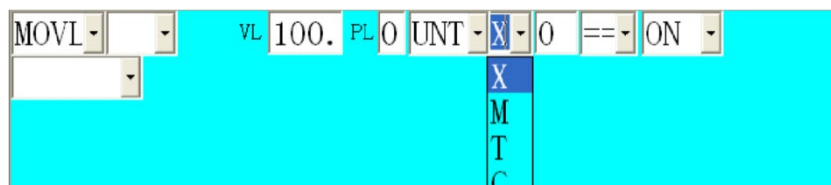
Movement instructions include three instructions: MOVJ、MOVL、MOVE

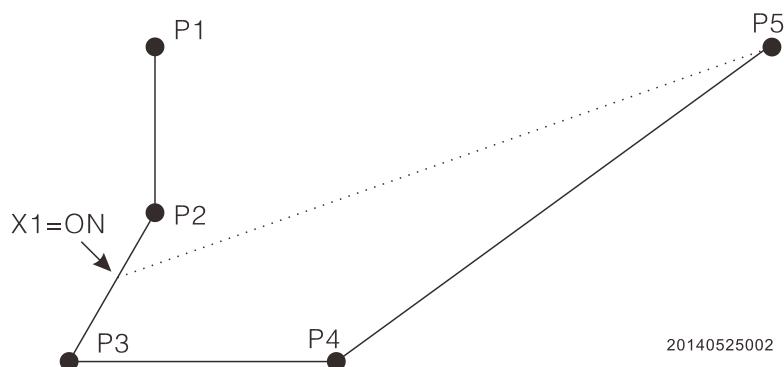
1Joint move (MOVJ)	functions	Move to the teach position in joint interpolation way. Movement speed of each joint is axis speed*VJ*automatic multiplier.	
	additions	[blank] GP<variable> LP< variable>	Position data, blank is shown in this bar on screen. GP variable, variable#: 0-999 LP variable, variable#: 0-999
		VJ=<%>	VJ speed percentage, percentage: 1%-100%.
		PL=<smoothness>	PL smoothness: 0-9
		[blank] UNTIL<condition>	Not available Condition. When condition is satisfied, the current line will be stopped, and the next will be executed. Detailed can be found in addition description 1.
		[blank] COORD COORD1 COORD2	Not available Added axis 1 and 2 is cooperated simultaneously Added axis 1 cooperated independently Added axis 2 cooperated independently Detailed refer to “chap9 1 cooperation manual”
	example	MOVJ VJ=30% PL=3 MOVJ VJ=30 GP#0 PL=3 UNTIL M#(0)==ON	

Addition description 1: example of UNTIL:

Execution condition. Then condition is satisfied, current program line will be stopped, and next line will be executed. Otherwise, execute current line before executing the next line. X, M, T, C can be used as condition.

This function could be used in digging, scooping aluminum, and plate loading.





position	line	program
P1	1	MOVJ VJ=30% PL=0
P2	2	MOVL VL=500MM/S PL=0
P3	3	MOVL VL=200MM/S PL=0 UNTIL X# (1) ==ON
	4	JUMP *22 IF X# (1) ==ON
P4	5	MOVL VL=200MM/S PL=0
	6	*22
P5	7	MOVJ VJ=30% PL=0

Notice:

After X1 is effective, system will jump out of the current 3# line, and execute the 7# line.

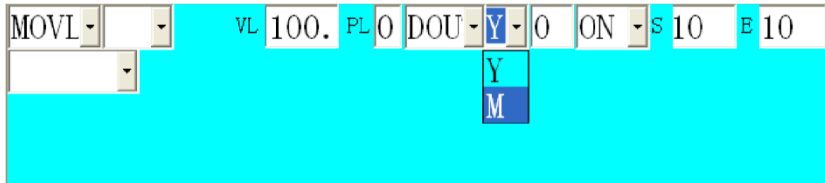
The effective time of X1 should be long, otherwise system will execute 5 lines.

2straight move (MOVL)	functions	Move to the teach position in a straight interpolation way. Each joint will move in straight interpolation way with the speed of VL*multiplier.	
	additions	[blank] GP<variable> LP< variable>	Position data, blank in this bar on the screen. GP variable, variable #: 0-999 LP variable, variable#: 0-999
		VL=<straight move speed>	Straight move speed, unit MM/S, max parameter of straight move highest speed.
		PL=<smoothness>	PL smoothness: 0-9
		[blank] UNTIL<condition> DOUT<addition> AOUT<addition>	Not available UNTIL: When condition is satisfied, stop executing the current line and move to next line. Details can be found in addition description 1 of MOVJ. DOUT: output signal in movement. Details can be found in addition description 2 of MOVL. AOUT: output analog index in movement. Details can be found in addition description 3 of MOVL.
		[blank] COORD COORD1 COORD2	Not available Added axis 1 and 2 is cooperated simultaneously Added axis 1 cooperated independently Added axis 2 cooperated independently Detailed refer to "chap9 1 cooperation manual"

	example	MOVL VL=500MM/S PL=0 MOVL VL=500MM/S GP#3 PL=5 UNTIL X# (0) ==ON MOVL VL=500MM/S PL=0 DOUT Y# (0) ==ON START 10.0 END 10.0 MOVL VL=500MM/S PL=0 AOUT A1=2.0 A2=2.0
--	---------	---

Addition description 2: DOUT example:

This addition is used to set a distance where the program should switch certain signal (M、Y)effective or ineffective, and to set a distance where restore that signal.

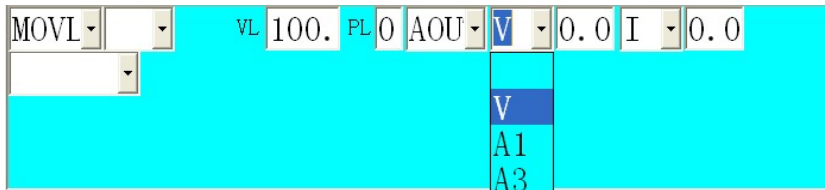


Example: MOVL VL=500MM/S PL=0 DOUT Y#(0)==ON START 10.0 END 10.0 Description: After executing this program, 10MM later, Y0 port change into open (ON). Robot keep moving to the point 10MM away from the end, and Y0 change to close (OFF). Then program come to the end.

This function can be used to open/close the gun in painting operation.

Addition description 3: AOUT example:

This addition is used to output the preset analog index in the movement or at the end point.



1)V I analog output:

VI analog index output can only be used among the welding instructions. If V1 is not among the welding instructions, when this line is executed, system will alart and prompt: no welding info, AOUT is not allowed to change the welding current and voltage during the movement.

V is correspondent to welding voltage, while I is correspondent to welding current. When this instruction is executed from the beginning to the end, analog voltage will change linearly from welding voltage to setting voltage.

Example:

Welding current 200A~analog index 4.95V, welding voltage 20V~ analog index 4.95V; Welding current 400A~analog index 9.95V, welding voltage 40V~analog index 9.95V.

ARCSTART# (0)

MOVL VL=500MM/S PL=0 AOUT V=40.0 I=400.0A

ARCEND# (0)

Description:

When 1stline is executed, A1 and A2 will output the arc start voltage. When 1st line is finished, A1 and A2 analog index output is 4.95V (Welding current and voltage).When 2nd line is finished, A1 and A2 analog index output change linearly from4.95Vto 9.95V. When 3rd line is executed, voltage become arc ending voltage.

This function can be used in the situation where welding current and voltage need changing.

A1 A2 analog output:

When this instruction is between the welding instructions, it's ineffective, and system will output according with the welding process settings.

When this instruction isn't between the welding instructions, system will output the A1 and A2 analog index at the end of this instruction line. This is called point output for short.

A3 A4 analog output:

No matter if this instruction is between the welding instructions or not, when system execute this instruction, robot will operate from the beginning to end, A3 and A4 analog index will change linearly from starting point to the end point. This is called linear output for short.

Example:

MOVJ VJ=30% PL=0 ; Point A A3=0 A4=0

MOVL VL=500MM/S PL=0 AOUT A3=10.0 A4=10.0 ; Point B

Description: Robot moves from point A to point B, and A3 and A4 analog index change linearly from 0V to 10V.

3.Circle move (MOVC)	functions	Move to the teach position in the circle interpolation way. Each joint will move in the circle interpolation with the speed of VL*automatic multiplier. The whole circle consists of two circular arcs, using at least four circular arc instructions. Details should refer to integrated example.	
	additions	[blank] GP<variable> LP<variable>	Position data, blank in the bar on the screen GP variable, variable#: 0-999 LP variable, variable#: 0-999
		VL=<straight move speed>	Straight move speed, unit MM/S, max is parameters of highest straight move speed.
		PL=<smoothness>	PL smoothness: 0-9
		<variable>POINT	Range: 1-3. One circular track must be realized by three circular instructions. Three circular instructions define the arc starting point, middle point, and end point. 1 POINT is the starting point of arc, 2 POINT is the middle point of arc, 3 POINT is the end point of arc.
		[blank] UNTIL<condition> DOUT<addition> AOUT<addition>	Not available UNTIL: When condition is satisfied, system will stop executing the current line and move to the next line. Details can be found in the MOVJ addition description 1. DOUT: output signal during movement. Details can be found in MOVL additions description 2. AOUT: output analog index during movement. Details can be found in MOVL additions description 3.
		[blank] COORD COORD1 COORD2	Not available Added axis 1 and 2 is cooperated simultaneously Added axis 1 cooperated independently Added axis 2 cooperated independently Detailed refer to "chap9 1 cooperation manual"

	example	[blank] COORD COORD1 COORD2	Not available Added axis 1 and 2 is cooperated simultaneously Added axis 1 cooperated independently Added axis 2 cooperated independently Detailed refer to “chap9 1 cooperation manual”
--	---------	--------------------------------------	--

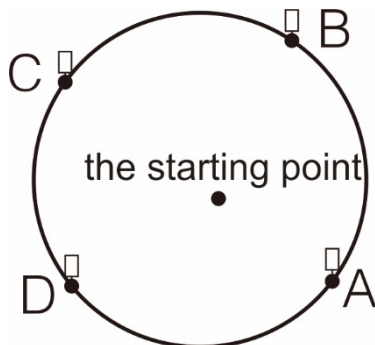
Integrated example of circular program without gesture:

Description
In order to move smoothly, please set the PL of every point (including starting point and every point on the arc) as 9.

Whole circle consists of ABC arc and CDA arc. Terminal of robot is located at the starting point.

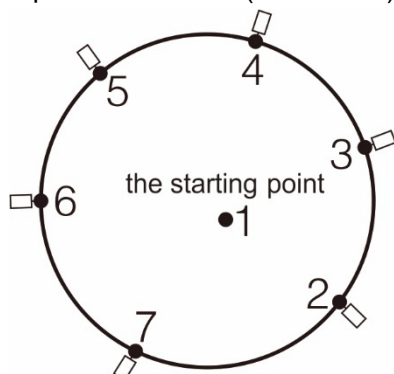
Program as follows:

MOVL VL=500MM/S PL=9	Move from starting point to point A, taking A as the first point on the arc.
MOVC VL=500MM/S PL=9 POINT=2	Move from A to B, taking B as the second point on the arc.
MOVC VL=500MM/S PL=9 POINT=3	Move from B to C, taking C as the third point on the arc and as the first point on the arc CDA.
MOVC VL=500MM/S PL=9 POINT=2	Move from C to D, taking D as the second point on the arc.
MOVC VL=500MM/S PL=9 POINT=3	Move from D to A, taking A as the third point on the arc.



Whole circle with gesture consists of three arcs.

6 points is needed (2 3 4 5 6 7) as follows:



MOVL VL=500MM/S PL=9	Move from starting point to point 2, taking 2 as the first point on the arc.
MOVC VL=500MM/S PL=9 POINT=2	Move from point 2 to 3, taking 3 as the second point on the arc.
MOVC VL=500MM/S PL=9 POINT=3	Move from point 3 to 4, taking 4 as the third point on the arc and as the first point on the next arc.

MOVCL=500MM/S PL=9 POINT=2	Move from 4 to 5, taking 5 as the second point on the arc.
MOVCL=500MM/S PL=9 POINT=3	Move from 5 to 6, taking 6 as the third point on the arc and as the first point on the next arc.
MOVCL=500MM/S PL=9 POINT=2	Move from 6 to 7, taking 7 as the second point on the arc.
MOVCL=500MM/S PL=9 POINT=3	Move from 7 to 2, taking 2 as the third point on the arc. At this time, whole circle with gesture is finished.

6.2 Logic Instructions

1 Digital quantity output (DOUT)	Functions	Control the status of variables. Only two types of index exists, i.e. “ON (effective)” and “OFF (ineffective)”.		
	Additions	Y<variable>= M<variable>=	ON OFF	Control the output port Y <variable>in ON or OFF status. Range of variables numbers should refer to “hardware manual” or “PLC manual”. Control the auxiliary relay M<variables>in ON or OFF status. Range of variables numbers should refer to “PLC manual”.
	example	DOUT Y#(0)=ON Control output port Y0 into ON status DOUT M#(0)=OFF Control the auxiliary relay M0 into OFF status		
2 Analog output (AOUT)	functions	Output the analog voltage of analog port.		
	additions	AO#<variable>=	Set the analog output port, range from A1-A2 in S40 and A1-A4 in S80.	
		<variable>	Variable is the output analog voltage, range0.000-10.000V	
	example	AOUT AO#(1)=0.000 output analog 0V voltage in A1 port. AOUT AO#(2)=10.000 output analog 10V voltage in A2 port.		
3 conditional wait (WAIT)	functions	When condition is satisfied, program will execute the following code. When condition is not satisfied, program will wait until condition is satisfied. However, there is a waiting time setting, when the condition is not satisfied but waiting time reaches the set time, program will move on.		
	additions	X<variable>== M<variable#>==	ON OFF	Judge if the input port X<variable> is ON or OFF. Range of variable number should refer to “hardware manual”. Judge of the auxiliary relay M<variable#>is ON or OFF. Range of variable number should refer to “PLC manual”.

		DT=<variable>		Instruction waiting time, units 1MS. DT = 0 means waiting the condition forever. DT ≠0 means whether condition is satisfied or time is reached, program will move on.							
		CT=<variable>		Signal last time, units 1MS.CT=0 means once condition is satisfied, program will move on. CT≠0 means condition need to last for a set time, then condition is regard as satisfied and program will move on.							
	example	WAIT X#(0)==ON T=0 Waiting X0 into On status forever. WAIT M#(1)==OFF T=500 Waiting M1 relay into ON status in 500MS, otherwise, program will move on.									
4 Delay (TIME)	functions	Waiting for a setting time (relay time), then program can move on.									
	additions	T=<variable>		Set the waiting (relay) time, units1MS, range: 1-99999999MS							
	example	TIME T=50 waiting for 50MS									
5 Pause (PAUSE)	functions	Pause without condition: pause until pressing “operation” button. Pause with condition: when following condition is satisfied, pause, or operate continuously.									
	additions	[blank]			Pause without condition						
		IF	X<variable#>		==	ON OFF	Pause with condition: If condition is satisfied, pause, otherwise, execue in order.				
			M<variable#>								
			Y<variable#>								
			T<variable#>								
			C<variable#>								
			GI<variable#>		<	GI<variable#> LI<variable#> GD<variable#> LD<variable#> GP<variable#> LP<variable#> TC<variable#> CC<variable#> D<variable#>					
			LI<variable#>								
			GD<variable#>								
			LD<variable#>								
			GP<variable#><data#>								
			LP<variable#><data#>					= =			
			TC<variable#>								
			CC<variable#>								
			GS<variable#>						<character string>		

	example	PAUSE PAUSE IF X#(0)==ON PAUSE IF GD#(1)==LD#(1)		Pause without condition If X0=ON, pause. Otherwise, continue. If GD1=LD1, pause. Otherwise, continue.		
6 condition jump (JUMP)	functions	Jump with condition and without condition. description: 1、 When using this instruction, star symbol (*) should be used to lable which line program need to jump to. Jump without condition: once program is executed here, jump to the labeled position. Jump with condition: when this line is executed, if condition is satisfied, program will jump to the labeled line. If condition is not satisfied, program will executed in order.				
				Jump to symbol: jump to the position labeled with *<variable>; variable can be any character or number.		
		*<variable>				
		[blank]		jump without condition		
	addition	IF	X<variable#>	==	ON OFF	Jump with condition: If condition is satisfied, jump to the position labeled with *<variable>. If condition is not satisfied, program will be executer in order.
			M< variable#>			
			Y< variable#>			
			T< variable#>			
			C< variable#>			
			GI< variable#>	>	GI< variable#> LI< variable#> GD< variable#> LD< variable#> GP< variable#> LP< variable#> TC< variable#> CC< variable#> D< variable#>	
			LI< variable#>			
			GD< variable#>	<		
			LD< variable#>			
			GP<variable#><data#>	>=		
			LP< variable#><data#>			
			TC< variable#>	<=		
			CC< variable#>			
			GS< variable#>	=		
	<character string>					

	example	1 * 345T : label *345T position 6 JUMP *SES IF M#(1)==ON : if M1=ON, jump to *SES position 12 JUMP *345T : Jump to *345T position without condition 13 *SES : label *SES position : Following lines				
7 call the sub program (CALL)	functions	Call the sub program, including call with condition and call without condition. descriptions: 1、 The difference of main program and sub program is that sub program is ended with RET. % is the subprogram users want to call. If no condition attached, program will call the sub program at this line. If condition is attached with call instruction, program will judge if the condition is satisfied. If satisfied, then call the subprogram. When using call without condition, there are fixed internal subprograms inside the robot to control the sliding table and spray gun (e.g. auto rotation 90, quick start, etc.).				
		%<program name>		name of sub program users want to call		
		[blank]		Call without condition. Call<program name >sub program when this line is executed.		
		IF	X<variable#>	==	ON OFF	Call with condition: If condition is satisfied, call the <program name> sub program. If condition is not satisfied, program will be executed in order.
			M< variable#>			
	Y< variable#>					
	T< variable#>					
	C< variable#>					
	GI< variable#>		>	GI<variable> LI <variable> GD<variable> LD<variable> GP<variable> LP<variable> TC<variable> CC<variable> D<variable>		
	LI< variable#>		<			
	GD< variable#>		> =			
	LD< variable#>		< =			
	GP< variable#><data#>		=			
LP< variable#><data#>						
TC< variable#>						
CC< variable#>						
GS< variable#>						
<character string>						
example	CALL 125 call the subprogram with name of 125 CALL %SBS IF LD#(1) > LD#(2) If LD1>LD2, call SBS sub program.					

8 notes (;)	function	Note for explain. When program comes to this line, notes will not be executed. Notes are used to help users to understand the program.	
	addition	; <notes>	Users can write the notes to check and understand the program.
	example	; tqzl : notes are tqzl, which will not be executed.	
9 jump symbol	function	Label the jump position. It should cooperate with jump instruction.	
	addition	*<variable>	Variable can be any character or number.
	example	* ssb4 : label jump position: *ssb4	
10 subprogram return 10 (RET)	function	RET instruction means the current program is sub program. System will return to main program when RET instruction is executed.	
	addition	none	
	example	 RET : return from sub program to main program	
11 output with time (TDOUT) 【shortcut: 2 1 1 E】	function	When this instruction is executed, after a certain period of time, Y or M will switch to the specific condition defined in the instruction.	
	addition	none	
	example	TDOUT Y#(0)=ON T=1000 : After 1S, Y0 port will switch to ON status.	

6.3 Calculation instructions

1 Add (ADD) 【shortcut: 3 1】	function	Add the former variable with later variable, and the result will be set to the former variable. E.g. : A=A+B. This instruction can only be used with number variables.		
	addition	GI<variable#> LI< variable#> GD< variable#> LD< variable#> GP< variable#> <data#> LP< variable#> <data#> TC< variable#> CC< variable#>	GI< variable#> LI< variable#> GD< variable#> LD< variable#> GP< variable#><data#> LP< variable#><data#> TC< variable#> CC< variable#> D< variable#>	Add the former variable with later variable, and the result will be set to the former variable.

	example	ADD TC#(4) GP#1(1) : TC4=TC4+GP1 x axis coordination ADD CC#(1) 20.5 : CC1=CC1+20.5		
2 subtraction (SUB) 【shortcut: 3 2】	function	Substract latter variable from the former one and the result will be setto the former variable. E.g.: A=A-B. This instruction can only be used with number variables.		
	addition	GI< variable#> LI< variable#> GD< variable#> LD< variable#> GP< variable#><data#> LP< variable#><data#> TC< variable#> CC< variable#>	GI< variable#> LI< variable#> GD< variable#> LD< variable#> GP< variable#><data#> LP< variable#><data#> TC< variable#> CC< variable#> D< variable>	Subtract latter variable from the former one and the result will be set to the former variable.
	example	SUB TC#(4) GP#1(1) : TC4=TC4-GP1 x axis coordination SUB CC#(1) 20.5 : CC1=CC1- 20.5		
3 multiplication (MUL) 【shortcut: 3 3】	Function	Multiply the former variable with the latter one and the result will be set to the former one. E.g.: A=A×B. This instruction can only be used with number variables.		
	addition	GI< variable#> LI< variable#> GD< variable#> LD< variable#> GP< variable#><dat a#> LP< variable#><dat a#> TC< variable#> CC< variable#>	GI< variable#> LI< variable#> GD< variable#> LD< variable#> GP< variable#><data#> LP< variable#><data#> TC< variable#> CC< variable#> D< variable>	Multiply the former variable with the latter one and the result will be set to the former one.
	example	MUL TC#(4) GP#1(1) : TC4=TC4×GP1 x axis coordination MUL CC#(1) 20.5 : CC1=CC1×20.5		
4 division (DIV)	function	Divide the former variable with the latter one and the result will be set to the former one. E.g.:		

【shortcut: 3 4】		A=A÷B。 This instruction can only be used with number variables.		
	additions	GI< variable#> LI< variable#> GD< variable#> LD< variable#> GP< variable#><data #> LP< variable#><data #> TC< variable#> CC< variable#>	GI< variable#> LI< variable#> GD< variable#> LD< variable#> GP< variable#><data#> LP< variable#><data#> TC< variable#> CC< variable#> D< variable>	Divide the former variable with the latter one and the result will be set to the former one.
	example	DIV TC#(4) GP#1(1) : TC4=TC4÷GP1 x axis DIV CC#(1) 20.5 : CC1=CC1÷20.5		
5 increasement +1 (INC) 【shortcut: 3 5】	functions	Add 1 to the variable, and the result will be set to the variable. E.g.: A=A+1 This instruction can only be used with number variables, and usually it's used to count.		
	additions	GI< variable#> LI< variable#> GD< variable#> LD< variable#> GP< variable#><data#> LP< variable#><data#> TC< variable#> CC< variable#>	Add 1 to the variable, and the result will be set to the variable.	
	example	INC TC#(4) : TC4=TC4+1 INC CC#(1) : CC1=CC1+1		
6 decrease -1 (DEC)	functions	Subtract 1 from the variable and the result will be set to the variable. E.g.: A=A-1. This instruction can only be used with number variables, and usually it's used to count.		

【shortcut: 3 6】	additions	GI< variable#> LI< variable#> GD< variable#> LD< variable#> GP< variable#><data#> LP< variable#><data#> TC< variable#> CC< variable#>	Subtract 1 from the variable and the result will be set to the variable.	
	example	DEC TC#(4) : TC4=TC4-1 DEC CC#(1) : CC1=CC1-1		
7 set (SET) 【shortcut: 3 7】	functions	Set the former variable value as the latter one. E.g.: A=B. This instruction can only be used with number variables		
	additions	GI< variable#> LI< variable#> GD< variable#> LD< variable#> GP< variable#> <data#> LP< variable#> <data#> TC< variable#> CC< variable#>	GI< variable#> LI< variable#> GD< variable#> LD< variable#> GP< variable#><data#> LP< variable#> <data#> TC< variable#> CC<variable#> D<data>	Set the former variable value as the latter one.
	example	SET TC#(4) GP#1(1) : TC4=GP1 X axis coordination SET CC#(1) TC#(1) : CC1=TC1 SET TC#(1) 5.000 : TC1=5		

6.4 Pallet instruction

1 Pallet (PALLET) 【shortcut: 4 E】	Function	Operate pallet process. When this instruction is operated, program will call the preset palletizing process. Variable number is the process number, range 0-9. Detailed palletizing infos can be found in "palletizing process manual".	
	addition	#<variable>	Variable is the palletizing process number users want to call.
	example	PALLET#(1) : Call palletizing process 1#	

6.5 Welding instructions

1Start arc (ARC START) 【shortcut: 5 1】	function	When this instruction is executed, system will call the preset welding parameters and start the arc. Variable number is the welding document number, range 0-7. The ARC START and ARC END are used in pair. Speed of program line will not be controlled by the auto multiplier. Only MOVL and MOVCL can be used between ARC START and ARC END. Detailed welding infos can be found in "Welding process manual".	
	addition	#<variable#>	Variable number is the welding parameter document number that user wants to call.
		Blank % MM/S	Method to treat the welding speed. Blank: Program between arc start and arc end is executed in 100% speed, without control of multiplier; %: Set the percentage of speed between arc start and arc end; MM/S: Set the execution speed of program between arc start and arc end. In this way, speed of program setting is not available any longer.
		V I	Set the welding current and voltage. Arc start and arc end is still according to the preset process, while welding current and voltage become the V I setting value.
	example	ARCSTART#(1) 8 MM/S V=20V I=200A ARCEND#(1)	Call welding parameter document 1#, start arc, welding speed 8 MM/S, welding voltage is 20V, welding current is 200A. starting welding Arc end, and welding finish.
2 end arc (ARC END) 【shortcut: 5 2】	function	When this instruction is executed, system will call the preset welding parameters and end the arc. Variable number is the welding document number, range 0-7. The ARC START and ARC END are used in pair. Detailed welding infos can be found in "Welding process manual".	
	addition	#<Variable#>	Variable number is the welding parameter document number that user wants to call.
	example	ARCSTART#(1) 50% ARCEND#(1)	Call the welding parameters document 1, start arc, welding program is executed with the speed of 50% of speed set by program. Start welding End the arc, welding finished.

Weave (WEAVE) 【shortcut: 5 3】 Weave end (WEAVEEND) 【shortcut: 5 4】	function	When this instruction is executed, system will call the preset welding weave parameters and weave. Variable number is the welding weave document number, range 0-7. The WEAVE START and WEAVE END are used in pair. Detailed welding infos can be found in “Welding process manual”.	
	addition	#<variable#>	Variable number is the welding parameter document number correspondent to the used weave instructions.
	example	WEAVE#(1) WEAVEEND#(1)	Call the weave document 1#, weave, weave track, Weave end

Integrated welding example:

ARCSTART #(1) 10MM/s V=20V I=200A	Call the 1# welding parameter document, speed 10MM/S, welding voltage 20V, welding current 200A
WEAVESINE #(1)	Call the 1# welding weave parameters
MOVL VL=100MM/S PL=0	Move along the welding track, speed is 10MM/S according to the ARCSTART
WEAVEEND	End weave
ARCEND #(1)	End the arc according to 1# welding process, end.

6.6 Painting Instruction (Backup Function)

6.7 Pouring Instruction (Backup Function)

6.8 Auxiliary Function

1 Speed change (SPEED) 【Shortcut: 8 E】	Function	This instruction is used to change the speed after this instruction by multiply the additional percentage with the program line speed. VJ<percentage> is effective to the VJ speed after this line. VL<percentage> is effective to the VL speed after this line. SPEED VJ=100or SPEED VL=100 can cancel the speed change function. This instructions mainly used to change the speed of certain part in the program. In replay mode, the operation multiplier is effective to the whole program.	
	Addition	VJ=<percentage> VL=<percentage >	Range: 1-100, effective to the VJ speed afterwards; Range: 1-100, effective to the VL speed afterwards;

	Example	SPEED VL=30 MOVL VL=500MM/S PL=0 SPEDE VL=100 MOVL VL=500MM/S PL=0	VL speed afterwards will multiply 30%.VLspeed change to: 500X30%=150MM/S. VL speed afterwards will multiply 100%.VLspeed change to: 500X100%=500MM/ S. (Equal to no speed change or cancel of speed change.)			
2 Condition (IF→IF) 【shortcut: 8 2 1】	Function	IF condition judgement instruction. The whole instruction should consist of four parts: IF, ELSEIF (can be omitted or be used repeatedly), ELSE, ENDIF. Firstly, judge if the condition is satisfied, if satisfied, the instruction after the IF will be executed; If not satisfied, judge the condition after ELSEIF (ELSEIF can be used according the situation or not used). If condition after ELSEIF is satisfied, the instructions after ELSEIF will be executed. If not satisfied, the instruction after ELSE will be executed. At the end of the IF condition judgement, use ENDIF to lable the end.				
	Addition	X<Variable#> M<Variable#> Y< Variable#> T< Variable#> C< Variable#>	== > < > = < = < = <	ON OFF	IF condition judgement instruction. If condition is satisfied, the instructions after the IF will be executed.	
		GI< Variable#> LI< Variable#> GD< Variable#> LD< Variable#> GP< Variable#><data#> LP< Variable#><data#> TC< Variable#> CC< Variable#>		GI< Variable#> LI< Variable#> GD<Variable#> LD<Variable#> GP<Variable#> LP<Variable#> TC<Variable#> CC<Variable#> D<Variable#>	If IF condition is not satisfied, judge the ELSEIF condition. If ELSEIF condition is satisfied, the instructions after the ELSEIF will be executed.If IF condition and ELSEIF	
		GS< Variable#>		<Character string>	condition(s) are not satisfied, the instructions after the ELSE will be executed. ENDIF to end the IF judgement.	

		<number>	Instruction structure label. In a complete structure, number should be accordant, otherwise, program will alart or jump out. The numbers used in one complete instruction sturture can be identical with the numbers used in another complete instruction structure. Range: 0-8		
	Example	IF X#(0)==ON 0 IF X0=ON, execute INC TC#(0) TC0=TC0+1 ELSEIF X#(1) = =OFF 0 IF X0=OFF, execute DEC TC#(0) TC0=TC0-1 ELSE 0 If above two conditions are not satisfied, ADD TC#(0) TC#(0) execute TC0=TC0+TC0 END IF 0End IF instruction			
2 condition (IF→ELSEIF) 【shortcut: 8 2 2】	Function	IF condition judgement instruction. The whole instruction should consist of four parts: IF, ELSEIF (can be omitted or be used repeatedly), ELSE, ENDIF. Firstly, judge if the condition is satisfied, if satisfied, the instruction after the IF will be executed; If not satisfied, judge the condition after ELSEIF (ELSEIF can be used according the situation or not used). If condition after ELSEIF is satisfied, the instructions after ELSEIF will be executed. If not satisfied, the instruction after ELSE will be executed. At the end of the IF condition judgement, use ENDIF to lable the end.			
	Addition	X<Variable#> M< Variable#> Y< Variable#> T< Variable#> C< Variable#>	==	ON OFF	IF condition judgement instruction. If condition is satisfied, the instructions after the IF will be executed.
		GI< Variable#> LI< Variable#> GD< Variable#> LD< Variable#> GP< Variable#><data#> LP< Variable#><data#> TC< Variable#> CC< Variable#>	> < > = < = =	GI< Variable#> LI< Variable#> GD<Variable#> LD<Variable#> GP<Variable#> LP<Variable#> TC<Variable#> CC<Variable#> D<Variable#>	If IF condition is not satisfied, judge the ELSEIF condition. If ELSEIF condition is satisfied, the instructions after the ELSEIF will be executed. If IF condition and ELSEIF condition(s)are not satisfied, the instructions after the
		GS< Variable#>		<Character string>	

					ELSE will be executed. ENDIF to end the IF judgement.
		<number>	Instruction structure label. In a complete structure, number should be accordant, otherwise, program will alart or jump out. The numbers used in one complete instruction sturture can be identical with the numbers used in another complete instruction structure. Range: 0-8		
	Example	IF X#(0)==ON 0 IFX0=ON, execute INC TC#(0) TC0=TC0+1 ELSEIF X#(1)==ON 0 IF X1=ON, execute ADD TC#(0) 2.000 TC0=TC0+2 ELSEIF X#(2)==ON 0 If X2=ON, execute ADD TC#(0) 3.000 TC0=TC0+3 ELSE 0 Otherwise, execute ADD TC#(0) TC#(0) TC0=TC0+TC0 ENDIF 0 0#IFinstruciton structure end			

2 condition judge (IF→ELSE) 【 shortcut: 8 2 3】	Function	IF condition judgement instruction. The whole instruction should consist of four parts: IF, ELSEIF (can be omitted or be used repeatedly), ELSE, ENDIF. Firstly, judge if the condition is satisfied, if satisfied, the instruction after the IF will be executed; If not satisfied, judge the condition after ELSEIF (ELSEIF can be used according the situation or not used). If condition after ELSEIF is satisfied, the instructions after ELSEIF will be executed. If not satisfied, the instruction after ELSE will be executed. At the end of the IF condition judgement, use ENDIF to lable the end.	
	Addition	<number>	Instruction structure label. In a complete structure, number should be accordant, otherwise, program will alart or jump out. The numbers used in one complete instruction sturture can be identical with the numbers used in another complete instruction structure. Range: 0-8
	Example	<pre>IF X#(1)==ON 0 If X1=ON, INC TC#(0) execute TC0=TC0+1. ELSE 0 Otherwise, ADD TC#(0) 3.000 execute TC0=TC0+3 END IF 00#IF instruction structure end IF X#(1)==OFF 0 If X1=OFF, ADD TC#(0) 4.00 execute TC0=TC0+4 ELSE 0 otherwise, ADD TC#(0) 5.00 execute TC0=TC0+5 END IF 00#IF instruction structure end</pre>	
2 Condition (IF→ENDIF) 【 shortcut: 8 2 4】	Function	IF condition judgement instruction. The whole instruction should consist of four parts: IF, ELSEIF (can be omitted or be used repeatedly), ELSE, ENDIF. Firstly, judge if the condition is satisfied, if satisfied, the instruction after the IF will be executed; If not satisfied, judge the condition after ELSEIF (ELSEIF can be used according the situation or not used). If condition after ELSEIF is satisfied, the instructions after ELSEIF will be executed. If not satisfied, the instruction after ELSE will be executed. At the end of the IF condition judgement, use ENDIF to lable the end.	
	Addition	<number>	Instruction structure label. In a complete structure, number should be accordant, otherwise, program will alart or jump out. The numbers used in one complete instruction sturture can be identical with the numbers used in another complete instruction structure. Range: 0-8

	Example	IF <condition><number> IF condition Program if IF condition is satisfied. ELSEIF < condition ><number> ELSE IF condition Program if ELSEIF condition is satisfied. ELSE<number> Program if the previous conditions are not satisfied. END IF <number> ENDIF to end IF structure				
3 FOR (backup)	Function	Backup instruction				
	Addition					
	Example					
4 While (WHILE →WHILE) 【 shortcut: 8 4 1】	Function	WHILE loop instruction. The complete structure should consist of WHILE and ENDWHILE. When WHILE condition is satisfied, i.e. WHILE condition is ON, the program between WHILE and ENDWHILE will be executed until WHILE condition is no longer satisfied. Then jump out of this loop. WHILE condition must be set inside the loop part, otherwise, endless loop may occur.				
	Addition	X<Variable#> M<Variable#> Y<Variable#> T<Variable#> C<Variable#>	==	ON OFF	WHILE loop condition instruction. When WHILE condition is satisfied, i.e. WHILE condition is ON, the program between WHILE and ENDWHILE will be executed. If WHILE condition is not satisfied, jump out of the loop, and execute the program following ENDWHILE.	
		GI< Variable#> LI< Variable#> GD< Variable#> LD< Variable#> GP< Variable#><data#> > LP< Variable#><data#> > TC< Variable#> CC< Variable#>	> < > = < = =	GI< Variable#> LI< Variable#> GD<Variable#> LD<Variable#> GP<Variable#> LP<Variable#> TC<Variable#> CC<Variable#> D<Variable#>		
		GS< Variable#>		<Character string>		
		<number>	Instruction structure label. In a complete structure, number should be accordant, otherwise, program will alart or jump out. The numbers used in one complete instruction sturture can be identical with the numbers used in			

			another complete instruction structure. Range: 0-8
	Example	SET TC#(1) 0.000 WHILE TC#(1) < 20 0 IF X#(1)==ON 0 INC TC#(1) ELSE 0 ADD TC#(1) 2.000 ENDIF 0 ENDWHILE 0 SET TC#(1) 15.000	SET TC1=0 IF TC1<20, Execute IF structure IF judge: if X1=ON, then TC1=TC1+1 Otherwise if X1≠ON, then TC1=TC1+2 IF structureend If TC1<20, execute TC1=15
4 while (WHILE →ENDWHILE) 【 shortcut: 8 4 2】	Function	WHILE loop instruction. The complete structure should consist of WHILE and ENDWHILE.	
	Addition	<number>	Instruction structure label. In a complete structure, number should be accordant, otherwise, program will alart or jump out. The numbers used in one complete instruction sturture can be identical with the numbers used in another complete instruction structure. Range: 0-8
	Example	WHILE <condition><number> ENDWHILE <number>	Judge condition, Satisfied: Executepart; Unsatisfied: Execute the program behind ENDWHILE.
5 switch (S W I T C H →SWITCH) 【 shortcut: 8 5 1】	Function	SWITCH condition selection instruction. The complete selection structure consists of SWITCH,CASE (can be used repeatedly),BREAK,DEFAULT, ENDSWITCH. Claculate the variable value behind the SWITCH, judge the value is equal to which value behind CASE. After finding the matched CASE, program will be executed from that line until the ENDSWITCH. If no identical value can be found, program behind DEFAULT will be executed until the ENDSWITCH. Notice: The different positions of BREAK may result into different result.	
	Addition	X<Variable#> M< Variable#> Y< Variable#> T< Variable#> C< Variable#>	Status variables, CASE= 0or 1

		GI< Variable#> LI< Variable#> GD< Variable#> LD< Variable#> GP< Variable#><data#> LP< Variable#><data#> TC< Variable#> CC< Variable#>	Number variables, CASE equal to specific number
		GS< Variable#>	Character string, CASE equal to specific character.
		<number>	Instruction structure label. In a complete structure, number should be accordant, otherwise, program will alart or jump out. The numbers used in one complete instruction sturture can be identical with the numbers used in another complete instruction structure. Range: 0-8
	example	SWITCH TC#(2) 0 CASE 10 0 AOUT AO#(1)=1.000 BREAK 0 CASE 20 0 AOUT AO#(1)=2.000 BREAK 0 CASE 30 0 AOUT AO#(1)=3.000 BREAK 0 CASE 40 0 AOUT AO#(1)=4.000 BREAK 0 CASE 50 0 AOUT AO#(1)=5.000 BREAK 0 CASE 60 0 AOUT AO#(1)=6.000 BREAK 0 CASE 70 0 AOUT AO#(1)=7.000 BREAK 0 CASE 80 0 AOUT AO#(1)=8.000 BREAK 0	case10 if TC2=10 output 1V vaoltage in A1 port case10 break to end switch case20 if TC2=20 output 2V voltage in A1 port case20 break to end switch case30 if TC2=30 output 3V voltage in A1 port case30 break to end switch case40 if TC2=40 output 4V voltage in A1 port case40 break to end switch case50 if TC2=50 output 5V voltage in A1 port case50 break to end switch case60 if TC2=60 output 6V voltage in A1 port case60 break to end switch case70 if TC2=70 output 7V voltage in A1 port case70 break to end switch case80 if TC2=80 output 8V voltage in A1 port case80 break to end switch case90 if TC2=90

		CASE 90 0 AOUT AO#(1)=9.000 BREAK 0 DEFAULT 0 AOUT AO#(1)=10.000 ENDSWITCH 0	output 9V voltage in A1 port case90 break to end switch if TC2 not equal to the above values, execute ouput 10V in A1 port. end 0# switch.
--	--	---	--

5 condition selection (S W I T C H → C A S E) 【 shortcut: 8 5 2】	Function	SWITCH condition selection instruction. The complete selection structure consists of SWITCH, CASE (can be used repeatedly), BREAK, DEFAULT, ENDSWITCH. Claculate the variable value behind the SWITCH, judge the value is equal to which value behind CASE. After finding the matched CASE, program will be executed from that line until the ENDSWITCH. If no identical value can be found, program behind DEFAULT will be executed until the ENDSWITCH.	
	Addition	<data>	Possible values behind SWITCH.
		<number>	Instruction structure label. In a complete structure, number should be accordant, otherwise, program will alart or jump out. The numbers used in one complete instruction sturture can be identical with the numbers used in another complete instruction structure. Range: 0-8
	example	SWITCH TC#(1) 0 CASE 1 0 CASE 2 0 CASE 3 0 CASE 4 0 CASE 5 0 DOUT Y#(18)==ON BREAK 0 DEFAULT 0 DOUT Y#(18)==OFF ENDSWITCH 0	SWITCHcalculate variable TC1 TC1=1/2/3/4/5, execute output effective in Y18 port (Y18=ON). CASE1-5 to ENDSWITCH TC1 not equal to the above value, execute output ineffective in Y18 port (Y18=OFF). SWITCHend.
5 switch (S W I T C H →DEFAUL T) 【 shortcut: 8 5 4】	Function	SWITCH condition selection instruction. The complete selection structure consists of SWITCH, CASE (can be used repeatedly), BREAK, DEFAULT, ENDSWITCH. Claculate the variable value behind the SWITCH, judge the value is equal to which value behind CASE. After finding the matched CASE, program will be executed from that line until the ENDSWITCH. If no identical value can be found, program behind DEFAULT will be executed until the ENDSWITCH.	
	Addition	<number>	Number range: 0-8.

	Example	SWITCH <variabel><number> CASE <value><number> BREAK <number> DEFAULT <number> ENDSWITCH <number>	
5 switch (S W I T C H → B R E A K) 【 shortcut: 8 5 5】	Function	SWITCH condition selection instruction. The complete selection structure consists of SWITCH, CASE (can be used repeatedly), BREAK, DEFAULT, ENDSWITCH. BREAK jump out of the current line to ENDSWITCH. Notice: The different positions of BREAK may result into different result.	
	Addition	<number> >	Number range: 0-8.
	Example	SWITCH <variable><number> CASE <value><number> BREAK <number> DEFAULT <number> ENDSWITCH <number>	
5 switch (S W I T C H →ENDSWI TCH) 【 shortcut: 8 5 6】	Funcitons	SWITCH condition selection instruction. The complete selection structure consists of SWITCH, CASE (can be used repeatedly), BREAK, DEFAULT, ENDSWITCH. End symbol of SWITCHstructure.	
	Additions	<number>	Number range: 0-8.
	Example	SWITCH <variable><number> CASE <value><number> BREAK <number> DEFAULT <number> ENDSWITCH <number>	
6 Pulse transform between axis (PULSETR ANSFORM)	Function	This complete instruction structure consists of PULSETRANSFORM andPULSETRANSRORMEND. Thisfunction will transfer the pluse from output axis to input axis, i.e. originally system sends pulse to output axis to move; after the execution of this instruction, pulse will be sent to input aixs, and input axis will move according to the input pluse, and output aixs will stop, and original input pluse will become ineffective. This instructionis effective to joint coordination.	

【 shortcut: 8 6】	Addition	<output axis#><input axis#>		Output axis is the axis which user wants to transfer pluse from; input axis is the axis which user wants to transfer pluse into. Axis number range: 1-8.
	Example	PULSETRANSFORM#(1)(2) PULSETRANSFORMEND		
7End of pulse transform between axis (PULSETRANSFORM END) 【 shortcut: 8 7】	Function	This complete instruction structure consists of PULSETRANSFORM andPULSETRANSRORMEND PULSETRANSRORMEND will end the pluse transform. And each axis will operate according to the pulse received by itself.		
	Addition	[none]		
	Example	PULSETRANSFORM#(1)(2) PULSETRANSFORMEND		
8 input e (IMPURED) 【 shortcut: 8 8】	Function	Treat the specific axis joint coordination: divide the coordinationby 360, and the remainder will be set as the current coordination. In this way, when zero return, joint will not rotate the whole angle. E.g.: J1 coordnaiton is 860.245. After inputing, coordination is 140.245. In zero return operation, J1 should rotate 140.245 rather than 860.245. This function is mainly used to the joints with incremental programmer.		
	Addition	<axis#>	Axis need imputing.range: 1-8	
	Example	IMPURED#(1) IMPURED#(2)	Impute J1 Impute J2	
9 zero return (ZERO) 【 shortcut: 8 9】	Function	This zero return instruction is specifically used for incremental programmer. When zero return instruction is executed, system will execute the mechanicalzero reture movement according the set aixs number and the zero return parameters of this axis.		
	Addition	<axis#>	Axis need zero return.range: 1-8	
	Example	ZERO #(1) ZERO #(2)	Zero return of axis 1 Zero return of axis 2	

1 0 Finish of motion (MOTION FINISH) 【 shortcut: 8 1 E】	Function	Motion finish is effective to the MOVJ, MOVL instructions before the motion finish line. Working process: The move instruction will move to the set position precisely. In motion finish process, system will compare the electrical machine feedback data with the setting data of program until two data is identical (i.e. when system pulse is about to finish, electric machine will also stop at the position, and position error is within the allowed limit.), MOTIONFINISH will end. Otherwise, when time is longer than the internal set time, system will prompt that motion finish time out.	
	Addition	[none]	
	Example	MOVL VJ=10% PL=5 MOTIONFINISH	Effective to the previous MOVL instruction to make sure the robot can move to the accurate position.
11 Change tool coordination (CHANGE USE) 【8 1 1 E】	Function	Change tool coordination instruction. When this instruction is executed, program will change from the current coordination number to the coordination set by CHANGEUSE. If change is from 0# coordination to other coordination or none-0# coordination to 0# coordination, there's no need to use this instruction. If change is from one none-0# coordination to another none-0# coordination, this instruction is needed. This operation may cause danger, please check the coordination number carefully and make sure there's no danger if robot works in the coordination.	
	Addition	<coordination#>	Number of coordination need changing, range: 0-49.
	Example	MOVJ VJ=10% PL=0 tool coordination 0# MOVJ VJ=20% PL=0 USE=3 change to 3# CHANGEUSE#(4) change to 4# MOVJ VJ=20% PL=0 USE=4 change to 4# MOVJ VJ=20% PL=0 change to 0#	
12 axis forbidden (FORBID AXIS) 【 shortcut: 8 1 1 1 E】 1 3axis forbidden end	Function	FORBIDAXIS and FORBIDAXISEND should be used in pair. Forbid the set axis to move, and cancel this forbid. Used in the situation where certain axis is not allowed to move. Later, this forbid may need removing. This instruction is effective to joint move, but ineffective to straight move. This instruction is effective in replay and remote mode, but ineffective in teach mode. Then switching to replay and remote mode, it's becomes effective again. Forbid of axis is ineffective after program is closed.	
	Addition	<axis#>	The axis that need forbidding move or removing the forbid. Range: 1-8~J1-J8axis.

(FORBID AXIS END) 【 shortcut: 8 ↑ ↑E】	Example	FORBIDAXIS#(1) :forbid J1 move FORBIDAXISEND# (1) :remove the forbid status of J1	
14 Coordinait on count (COUNT COORD) 【 shortcut: 8 ↑ ↑E】	Function	When this instructionis executed, system will read the current axis programmer data, and then calculate the current coordination.	
	Addition	parameters: 0or 1	0: Coordination calculation time out or beyond the error range, system will alert. 1: Coordiantion calculation operating, no alarm.
	Example	COUNTCOORD(0) Read all axis programmer data, calculate the coordination, and alert if any error. COUNTCOORD(1) Calculate coordination without alarm.	

6.9 Vision Instruction

1 vision run (RUN VISION) 【 shortcut: 1 1 E 1】	Function	Vision run instruction: system will call the process number related parameters, and establish the connection with the system. If process document has start based on time or touch, after this instruction, vision system will start continous start according the preset. Data will be stored in the system. If connection process is during the program execution, connection process will continue until program finish and quit. If there are many vision-run instructions in one program, only the first one is effective, i.e. when program is executed continuously, the first execution of vision run instruction is effective and the rest instructions will not be effective.	
	Addition	<process#>	Vision document number, range 0-9. One process document is correspondent to one set of vision system.
	Example	RUNVISION# (0) Establish the connection of system and vision with the parameters in 0# process document.	
2 Get vision data (GET VISON DATA) 【 shortcut: 1 1 E 2】	function	Import vision cache data into the GP52 and GP52 (the values of GP52 and GP52 is same.).	
	addition	<process#>	Vision document number, range 0-9. One process document is correspondent to one set of vision system.
	example	GETVISONDATA #(0) Import data into GP52 and GP53.	

3 Clear vision data (CLEAR VISION DATA) 【shortcut: 1 1 E 3】	function	Clear the vision data in current cache area.	
	addition	<process#>	Vision document number, range 0-9. One process document is correspondent to one set of vision system.
	example	CLEARVISIONDATA#(0) Clear the tool data acquired by 0# process camera in the cache.	
4 vision trigger (TRIGGER VISION) 【shortcut: 1 1 E 4】	function	When trigger instruction is chosen in certain process document, this instruction can trigger the vision system, calculate the tool data, write the acquired data into vision cache, and write the number of data into GI50. In order to get rid of delay, this instruction will directly trigger the vision system via output port Y.	
	addition	<process#>	Vision document number, range 0-9. One process document is correspondent to one set of vision system.
	example	TRIGGERVISION#(0) trigger the 0# process vision system	
5 Camera program change (CHANGE VISION) 【shortcut: 1 1 E 5】	function	When this instruction is executed, system will change the tool calculation parameters document by the data connect wire to the vision system.	
	addition	<documentname>	The document userd want to switch into. The system only supports number name, so document in vision system need naming by number.
	example	The calculation parameter document related to tool A is 111, while the calculation parameter document related to tool B is 222. Currently, vision system is executing the tool A document. CHANGVISION# (222) Vision system will call the tool B calculation parameters document (222).	

Integrated vision example:

RUNVISION#(0)	run 0# vision process
*22	jump symbol *22
TRIGGERVISION#(0)	Trigger for first time, acquir camera calculation data of tool (相机计算工件数据).
TIME T= 200	delay 200ms
JUMP *22 IF GI#(50)<=0.000	Judge if GI50 has data. If no, jump to *22, trigger again.
GETVISIONDATA#(0)	Acquir data from vision cache, write them into GP52 and GP53.
ADD GP#53(3) 10.000	Move GP53 for 10MM in Z direction, in case of collision.

MOVL VL=500MM/S GP#53 PL=0	Move to GP53 safe point.
MOVL VL=100MM/S GP#52 PL=0	Move to GP52 grab point (抓取点).
DOUT Y#(1)=ON	Grabe
MOVL VL=500MM/S GP#53 PL=0	Move to GP53 safe point.
.....	Following release track and move.

6.10 Track instructions

1 Strat track(TARCK START) 【 shortcut: 1 1 1 E 1】	function	When this instruction is executed, system will call the track process parameters, write the the current tool position of cache into the GP50 and GP51 (the values of GP52 and GP52 is same). At this time, the data in GP50 and GP51 is real-time changing. Then system will call the GP50 and GP51 data to start tracking.	
	addition	<process#>	Tracking document number, range 0-9.
	example	TRACKSTART#(0) Prepare the GP50 and GP51 data	
2 End track(TARCK END) 【 shortcut: 1 1 1 E 2】	function	Track end, robot will stop tracking tool. Data of GP50 and GP51 will not be real-time any longer.	
	addition	<process#>	Tracking document number, range 0-9.
	example	TRACKEND# (0) End of tool tracking.	
3Acquir track data(GET TRACK DATA) 【 shortcut: 1 1 1 E 3】	function	Call the track document set parameters, judge the data in tracking cache. If no data in tracking cache, wait. If tool hasn't entered the AB region, wait. If tool has gone beyond the B, abandon the data and wait.	
	addition	<process#>	Tracking document number, range 0-9.
		X<data> Y< data > Z< data >	Compensation data for each axis. If no compensation is need, set the data into 0.
	example	GERRTACKDATA# (0) X# (0.0) Y# (0.0) Z# (0.0) Judge if the tracking cache has effective data. No compensation for XYA axis.	
4 clear track data	function	Clear the tracking cache data related with the set tool number. This instruction is used in the cache cleanup situation.	

(CLEAR STACK DATA) 【 shortcut: 1 1 1 E 4】	addition	<process#>	Tracking document number, range 0-9.
	example	CLEARTRACKDATA#(0) Clear the 0# tracking document-related data in cache.	
5back stage IO detection (RUN IO CUTIN) 【 shortcut: 1 1 1 E 5】	function	When this instruction is executed, IO detection point will detect if there is object pass through the detection point. If object passing, write the current data into cache for further call.	
	addition	<process#>	Tracking document number, range 0-9.
	example	RUNIOCUTIN# (0) IO detection starts in backstage. Detection point will detect continuously.	

Integrated tracking example:

MOVL VL=500.00MM/S PL=0	Move to the air above the middle point, waiting for grab.
RUNIOCUTIN #(1)	Start backstage IO detection. If there is object passing, the current position data will be written into cache.
GETTRACKDATA#(1) X#(0.0) Y#(0.0) Z#(0.0)	Judge the data in tracking cache. If no data in tracking cache, wait. If tool hasn't entered the AB region, wait. If tool has gone beyond the B, abandon the data and wait.
TREACKSTART #(1)	Track start; write the current tool data of cache into GP50 and GP51.
ADD GP#51(3) 10.000	Move GP51 for 10mm in Z direction, set this point as safe point. The meaning of (3) can refer to the data single-axis operation of GP variable.
MOVL VL=500MM/S GP#51 PL=0	Move to GP51 safe point.
MOVL VL=100MM/S GP#50 PL=0	Move to GP50 tracking point.
DOUT Y#(0) =ON	Output Y0 is effective; grab objective.
MOVL VL=500MM/S GP#51 PL=0	Lift the objective, move to GP51 safe point.
TRACKEND #(1)	Track end.
.....	Following operation.

6.11 Communication instructions

	function	Read the status of input port X, and write the data into set variable.
--	----------	--

1Read BCD data of IO port 【shortcut: ↑ ↑ E 1】	addition	GI<variable> X<port> B<number>	Data storage variable. Read the initial position of X input port. Read the number of X ports.
	example	READIOBCD #(20) #(0) #(4) Read the status data of X0-X3 (4 ports) into GI20 variable.	
2read data to (READ DATA TO) 【shortcut: ↑ ↑ E 2】	function	Communicate the data to the latter variable through serious port. Communication port # and communication fuction should bet in operation parameters. This function is mainly to read barcode, code, etc.	
	addition	GI<variable#> GD< variable#> GS< variable#>	Communicate data to GI variable. Communicate data to GD variable. Communicate data to GS variable
	example	READDATATO GI# (20) Read the port data into variable GI20.	

6.12 Special Instructions

1reset(RESET) 【shortcut: ↑ E 1】	function	When this instruction is executed, system will return backwards to the first line of the program line to line. Even if current execution is within the subprogram, system will quit the subprogram and return to the first line of main program after this instruction is executed.	
	addition	[none]	
	example	1 CALL 123 ----- 1 RESET RET	Main program XX calls 123 subprogram Lines after subprogram is end -----subprogram123 program reset subprogram return
2 Program pointer jump (CHANGEPOINT) 【shortvut: ↑ E 2】	function	Move the automatical line pointer when operation. E.g. line pointer stops at 18 th line in automatic operation. Afet the use of CHANGEPOINT# (-2), the program stop line will jump to 18-2=16 th line.	
	addition	<line number>	The lines of jump users want. Positive value means jump backwards, and negative value means jump forwards.
	example	CHANGEPOINT#(-2)	

		Stop line move 2 lines afterwords. E.g. line pointer stops at 18 th line in automatic operation. After the use of CHANGEPOINT# (-2), the program stop line will jump to 18-2=16 th line.
--	--	--

7 Teach program

Teach program is the process of creating a user program in teach mode, including selecting correct coordination system, manually move the terminal of robot into the desired position; call the current coordination data, operation track, and process instructions by certain operation (e.g. press button to select the instructions, coordination data is invisible).

Movement instruction line includes move instruction, additional speed, smoothness, and joint data of robot (joint data is invisible in the data program edit interface).

If variables are used, the shutdown data of robot is contained in the variables.

The following sections are the examples to describe the teach program steps. For more program example, please refer to "process manual".

This system supports the Chinese character display of program.

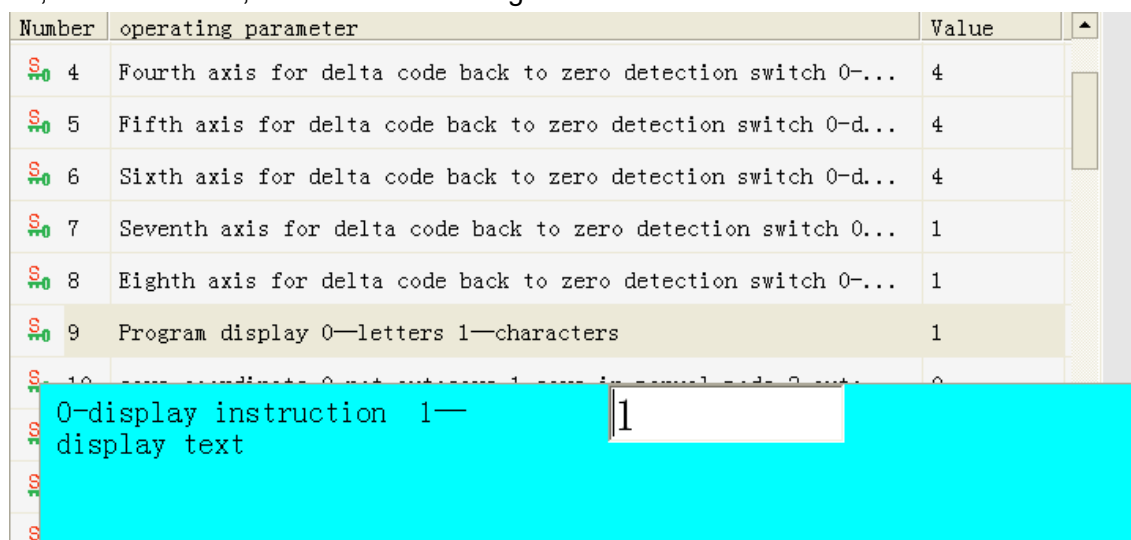
Correspondent parameters:

parameter type	parameter	value	description
operation parameters	program display	0	Letters: program is shown with letters.
		1	Characters: Program is shown with Chinese characters.

Specific steps:

Firstly, <parameter settings>-<7 system parameters>-<1 operation authority selection>-<confirm>. Input the integrator password in the prompt interface, click <confirm>, the authority will be changed to integrator authority.

Then, < parameter settings >-<3 operation parameters>-<confirm>. Choose [program display] using cursor in the prompt program list. Click <modify> in the sub menu. Input number 1 in the prompt input window, click <confirm>, as shown in below figure.



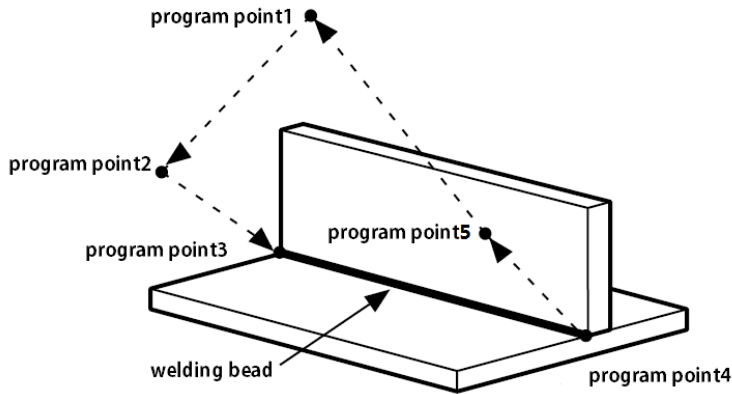
At this time, teach program in the system will be displayed in Chinese character, as shown in below:

```

1  关节运动 关节速度=10.0% 平滑=0
2  直线运动 直线速度=100.0MM/S 平滑=0
3  圆弧运动 直线速度=100.0MM/S 平滑=0 点=1
4  输出 DOUT Y#(0)=ON
5  子程序调用 CALL 111
6  延时 TIME T=500
7  等待信号 WAIT X#(0)=ON T=0
8  0
  
```

7.1 Welding teach program

Scheme as follows:



related instructions:

MOVJ: joint move

MOVL: Line move

VJ: speed multiplier of joint move

VL: Speed

multiplier of straight move

PL: smoothness

USE: tool coordination

ARCSTART: arc strat

ARCEND: arc end

Program list

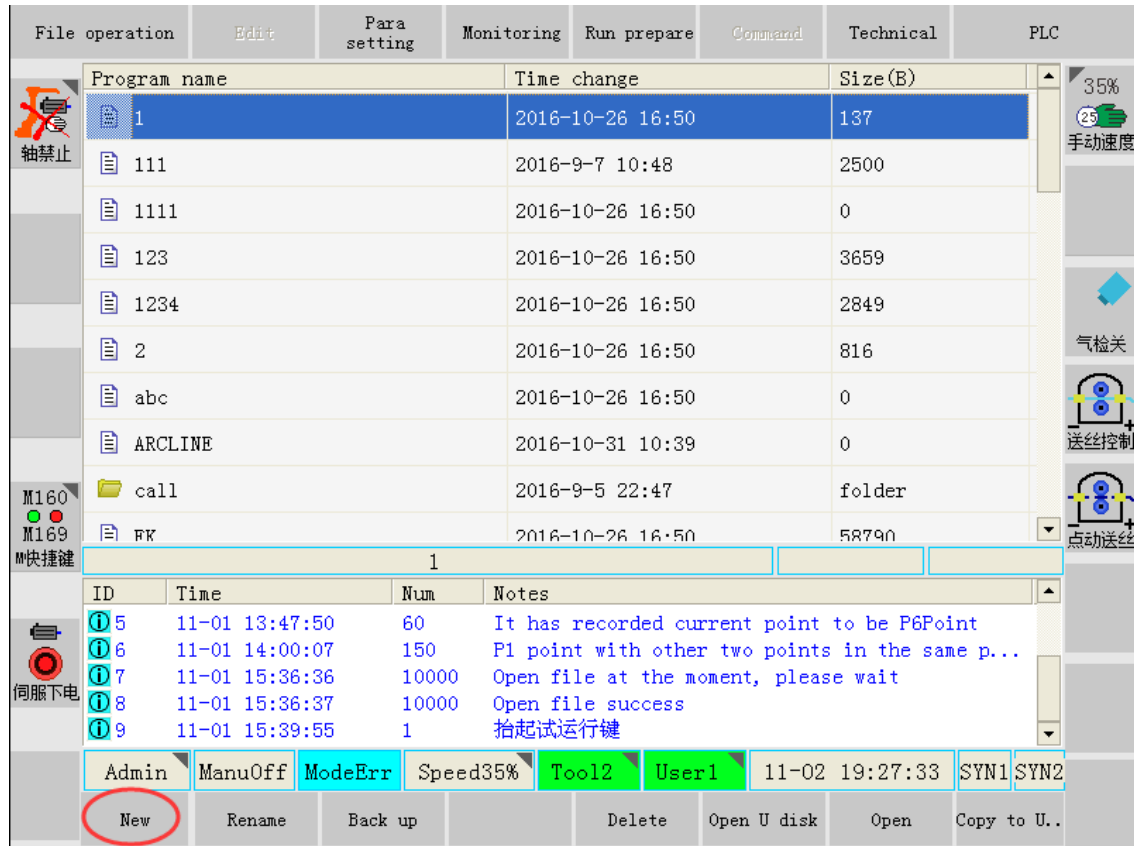
program instructions	descriptions
MOVJ VJ=50% PL=9 USE=1	In user coordination syetem USE=1, MOVJ joint move, VJ=50 %(multiplier), PL=9(smoothness), move to program point 1, reach the prepare point.
MOVJ VJ=50% PL=9 USE=1	In user coordination syetem USE=1, MOVJ joint move, VJ=50 %(multiplier), PL=9(smoothness), move to program point 2, approach to objective.
MOVJ VJ=25% PL=0 USE=1	In user coordination syetem USE=1, MOVJ joint move, VJ=25 %(multiplier), PL=0(smoothness), move to program point 3, touch the objective.
ARCSTART#(0)	arc start
MOVL VL=100.0MM/S PL=0 USE=1	In user coordination syetem USE=1, MOVL joint move, VL=100MM/S, PL=0(smoothness), move to program point 4, operate in welding track.
ARCEND#(0)	arc end
MOVJ VJ=50% PL=9 USE=1	In user coordination syetem USE=1, MOVJ joint move, VJ=50 %(multiplier), PL=9(smoothness), move to program point 5, depart from the objective.
MOVJ VJ=50% PL=9 USE=1	In user coordination syetem USE=1, MOVJ joint move, VJ=50 %(multiplier), PL=9(smoothness), move to program point 1, return to prepare point.

Teach program steps

1. Switch to teach mode.

2. Choose proper coordination system.

3. Enter the program list interface.



4. Click **New** button.

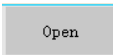
5. Input the new program name (e.g. 555555) in the prompt interface. After the input, click <confirm>.

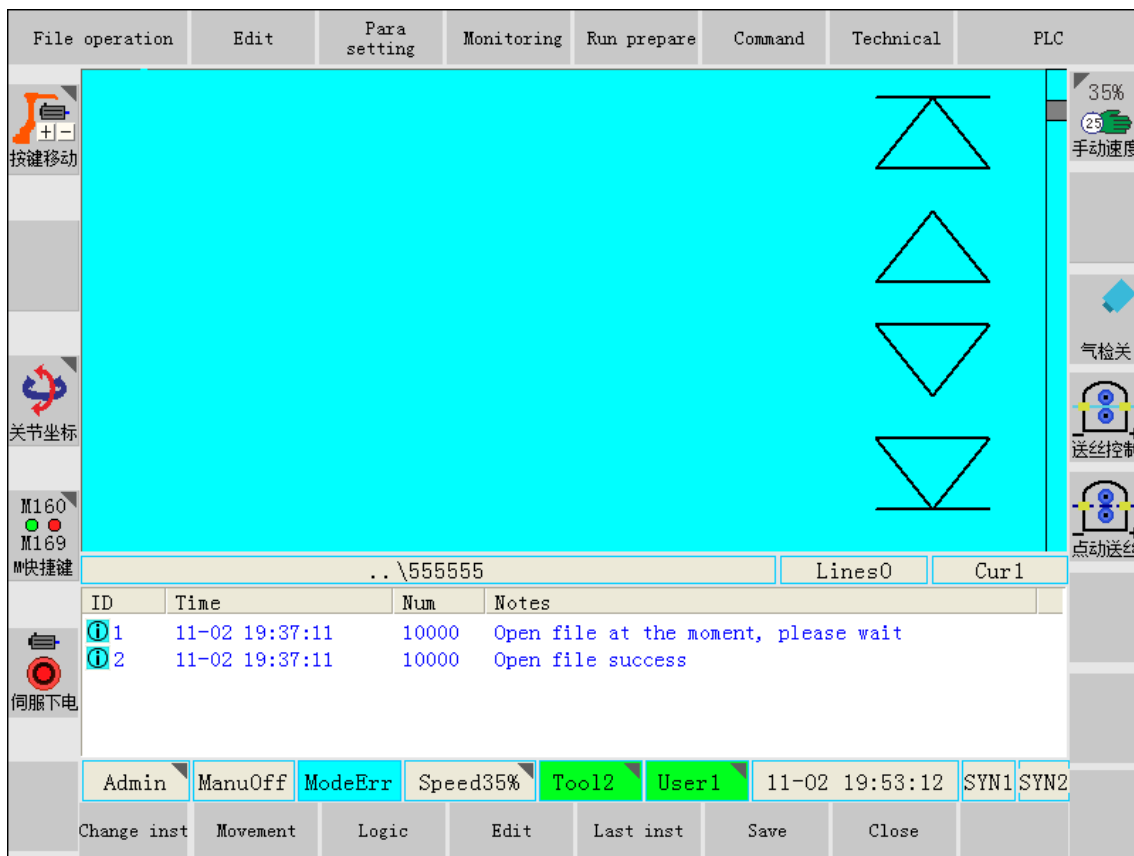
At this time, the new program will shown in the program list interface, as follows:

Description
If input the new program name, system will create a new program and highlight it by blue bar. If input the already-exist program name, system will highlight the program.

New name

Program name	Time change	Size (B)
1	2016-10-26 16:50	137
111	2016-9-7 10:48	2500
1111	2016-10-26 16:50	0
123	2016-10-26 16:50	3659
1234	2016-10-26 16:50	2849
2	2016-10-26 16:50	816
555555	2016-11-2 19:35	0
abc	2016-10-26 16:50	0
ARCLINE	2016-10-31 10:39	0
call	2016-9-5 22:47	folder

6. Click submenu  button, open program edit window:



The screenshot displays the software interface with the following components:

- Top Menu Bar:** File operation, Edit, Para setting, Monitoring, Run prepare, Command, Technical, PLC.
- Left Sidebar:**
 - Buttons: 按键移动 (Move), 关节坐标 (Joint Coordinate), M160, M169, M快捷键 (M Shortcut Key), 伺服下电 (Servo Power Off).
- Main Area:** A large cyan rectangle representing the program edit window. To its right are four triangular icons (up, down, left, right) and a vertical slider.
- Right Sidebar:**
 - Buttons: 35% (Manual Speed), 气检关 (Air Check Off), 送丝控制 (Wire Feeding Control), 点动送丝 (Jog Wire Feeding).
- Bottom Panel:**
 - Path: .. \555555
 - Fields: Lines0, Cur1
 - Log Table:

ID	Time	Num	Notes
1	11-02 19:37:11	10000	Open file at the moment, please wait
2	11-02 19:37:11	10000	Open file success
 - Buttons: Admin, ManuOff, ModeErr, Speed35%, Tool2, User1, 11-02 19:53:12, SYN1, SYN2.
 - Bottom Bar: Change inst, Movement, Logic, Edit, Last inst, Save, Close.

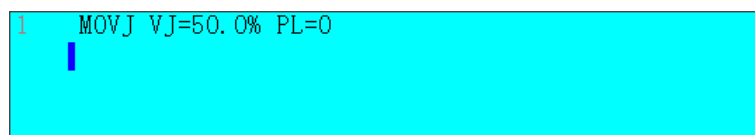
7. Holding security switch, move the terminal of robot into program point 1 by coordination button on the teach box. Click <Movement>-<MOVJ>.



Prompt instruction edit window.



Input related parameters, press **Inst correct**. Instruction will show in program edit window.

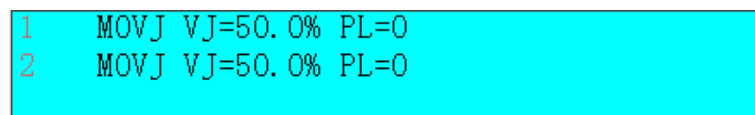


Finish of instruction edit of program point 1.

8. Holding security switch, move the terminal of robot into program point 2 by coordination button in the teach box. Click<Movement>-<MOVJ>. MOVJ instruction edit window will prompt as follows:



Input related parameters, press **Inst correct**. Instruction will show in program edit window.



Finish of instruction edit of program point 2.

9. Repeat step 7 or 8, change VJ speed to 25%, PL=0. Input the instruction line of program point 3.

MOVJ VJ=25.0% PL=0

```

1  MOVJ VJ=50.0% PL=9 TOOL=1
2  MOVJ VJ=50.0% PL=9 TOOL=1
3  MOVJ VJ=25.0% PL=0 TOOL=1
4

```

10. Click<program instruction>-<5 welding>-<1 ARC START>-<confirm>, following window will prompt.

ARCSTART 0

Input related parameters, press Inst correct, Instruction will show in program edit window.

```

1  MOVJ VJ=50.0% PL=9 TOOL=1
2  MOVJ VJ=50.0% PL=9 TOOL=1
3  MOVJ VJ=25.0% PL=0 TOOL=1
4  ARCSTART#(0)
5

```

Input related parameters, press Inst correct Instruction will show in program edit window.

```

1  MOVJ VJ=50.0% PL=0
2  MOVJ VJ=50.0% PL=0
3  MOVJ VJ=50.0% PL=0
4  ARCSTART#(0)

```

11. Repeat the above steps, and input the every program points and instructions as follows:

```

1  MOVJ VJ=50.0% PL=0
2  MOVJ VJ=50.0% PL=0
3  MOVJ VJ=50.0% PL=0
4  ARCSTART#(0)
5  MOVL VL=100.0MM/S PL=0
6  ARCEMEND#(0)
7  MOVJ VJ=50.0% PL=0
8  MOVJ VJ=50.0% PL=0

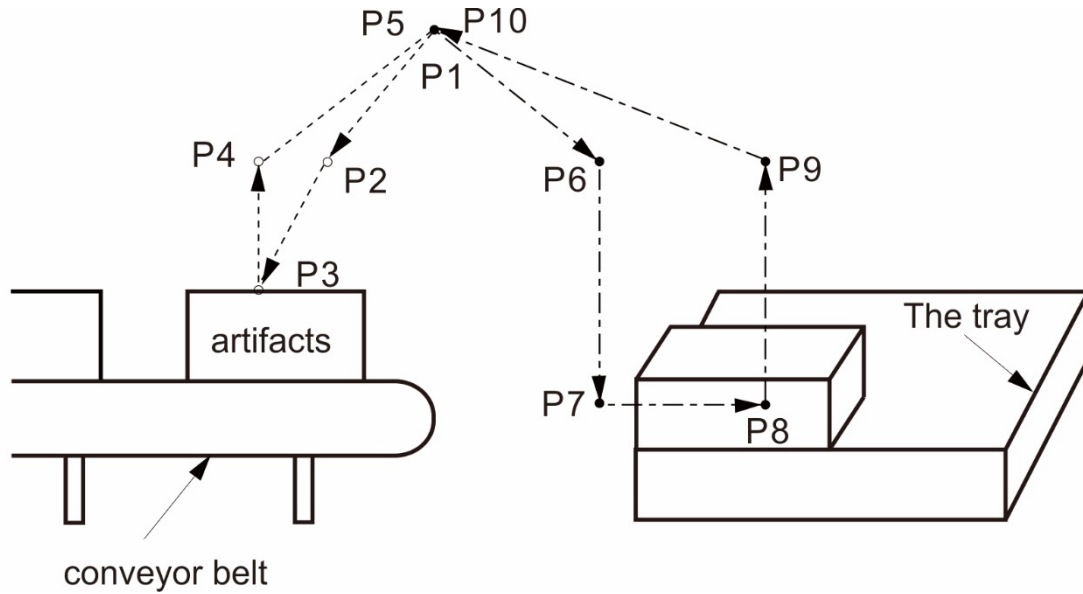
```

The welding program is created by the above steps.

12 Click sub menu Save, then Close, close program edit window.

7.2 Lifting program example

scheme as follows:



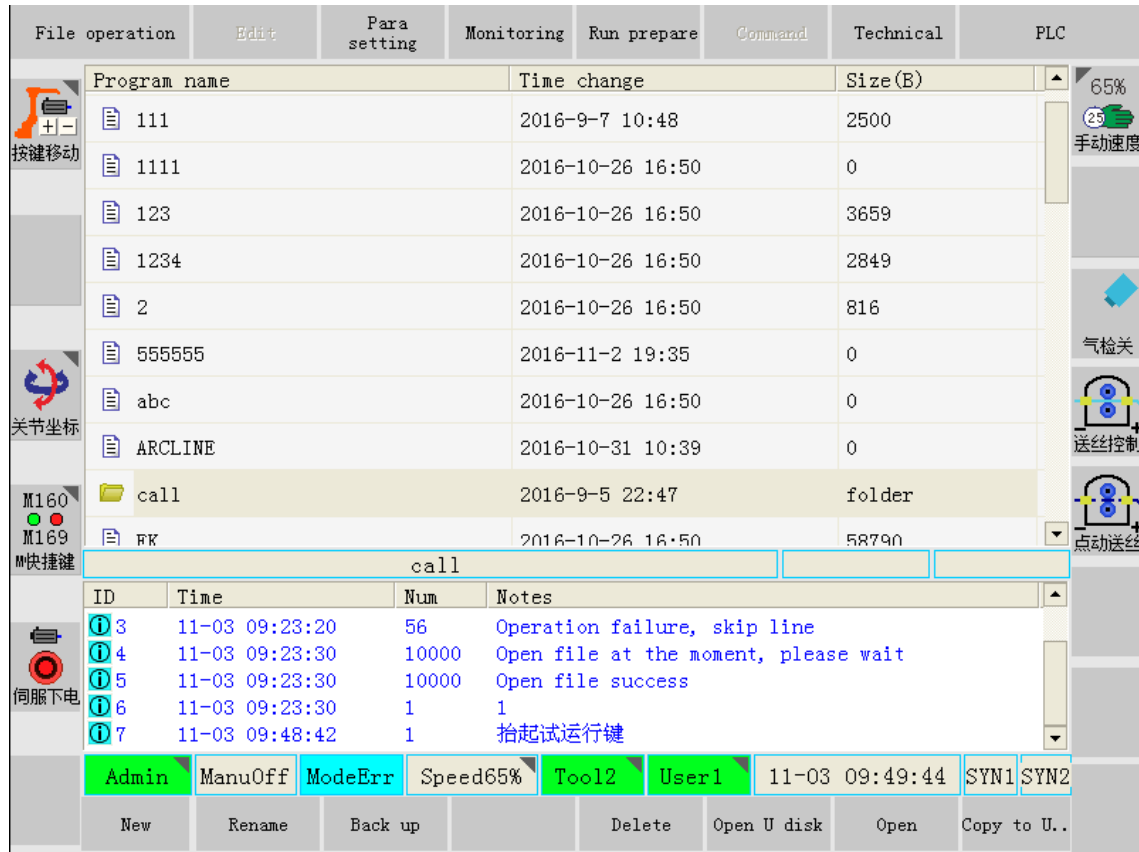
MOVJ: joint move MOVL: joint move VJ: speed multiplier of joint move VL: Speed multiplier of straight move
PL: smoothness USE: tool coordination DOUT: number index output
WAIT: conditional wait

7.2.1 program list

program instructions	descriptions
MOVJ VJ=50% PL=9 USE=1	In user coordination syetem USE=1, MOVJ joint move, VJ=50% (multiplier), PL=9(smoothness), move to program point 1, reach the prepare point.
MOVJ VJ=50% PL=9 USE=1	Move to program point 2, approach to objective (before grab)
MOVL VL=100MM/S PL=0 USE=1	Move to program point 3, touch the objective (grab position)
DOUT Y#(0)=ON	grab
WAIT X#(0)=ON T=0	detect if grab is finished
MOVL VL=200MM/S PL=9 USE=1	Move to program point 4, leave objective (after grab)
MOVJ VJ=50% PL=9 USE=1	Move to program point 5, back to initial position
MOVJ VJ=50% PL=9 USE=1	Move to program point 6, approach put position
MOVJ VJ=50% PL=9 USE=1	Move to program point 7, reach auxiliary put point
MOVL VL=100MM/S PL=0 USE=1	Move to program point 8, reach put position
DOUT Y#(0)=OFF	put
WAIT X#(0)=OFF T=0	detect if put is finished
MOVL VL=200MM/S PL=9 USE=1	Move to program point 9, leave put position
MOVJ VJ=50% PL=9 USE=1	Move to program point 10, back to initial position

7.2.2 Steps for teach program

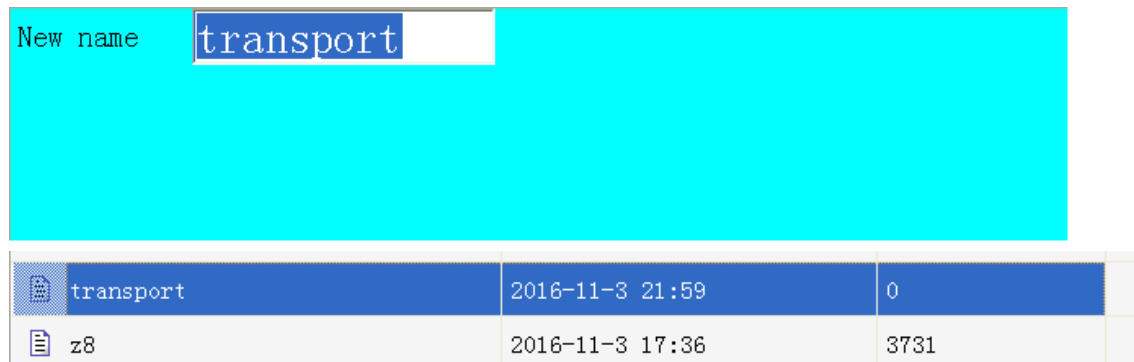
1. Switch to teach mode.
2. Choose proper coordination system.
3. Enter the program list interface.



4. Click the  button.

5. Input the new program name (e.g. transport) in the prompt interface. After the input, click <confirm>. At this time, the new program will show in the program list interface, as follows:

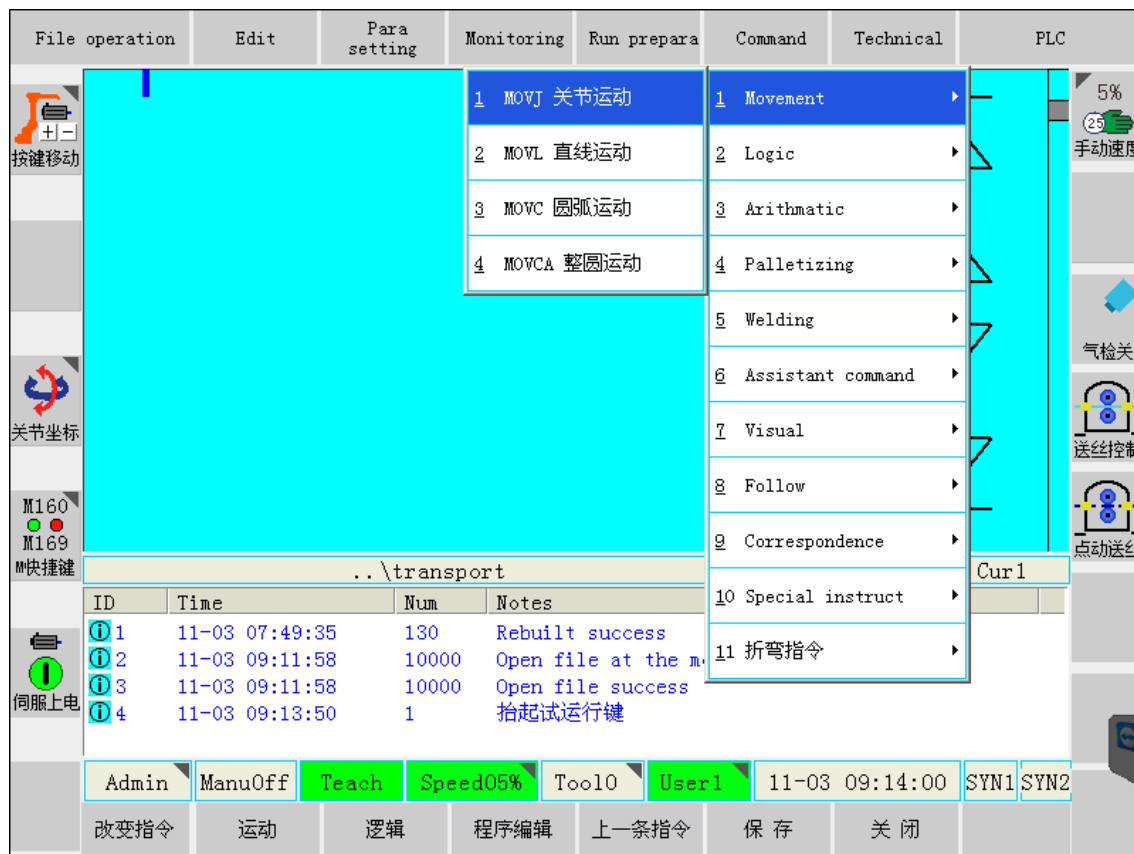
Description
If input the new program name, system will create a new program and highlight it by blue bar. If input the already-exist program name, system will highlight the program.



6. Click sub menu  button, open program edit window.



7. Holding security switch, move the terminal of robot into program point 1 by coordination button on the teach box. Click <Command>-<Movement>-< MOVJ>-



Prompt instruction edit window.

MOVJ VJ= 50.0 % PL= 9

Description

VJ= 50.0 Is the input window, data can be written into it.

Input related parameters, press **Inst correct**. Instruction will show in program edit window.

```
1  MOVJ VJ=50.0% PL=0
```

Finish of instruction edit of program point 1.

8. Holding security switch, move the terminal of robot into program point 2 by coordination button in the teach box. Click <Command><1 MOVJ>. MOVJ instruction edit window will prompt as follows:

MOVJ VJ= 50.0% PL= 9

Inst correct

Input related parameters, press . Instruction will show in program edit window.

```
1  MOVJ VJ=50.0% PL=0
2  MOVJ VJ=50.0% PL=0
```

Finish of instruction edit of program point 2.

9. Repeat step 7 or 8, change VJ speed is 100MM/S, PL=0. Input the instruction line of program point 3.

MOVL VL 100 PL 0

```
1  MOVJ VJ=50.0% PL=0
2  MOVJ VJ=50.0% PL=0
3  MOVL VL=100.0MM/S PL=0
```

10. Click<Command>-<2 Logic>-<1 DOUT>, following window will prompt.

DOUT Y 0 = ON

Input related parameters, press . Instruction will show in program edit window.

```
1  MOVJ VJ=50.0% PL=0
2  MOVJ VJ=50.0% PL=0
3  MOVL VL=100.0MM/S PL=0
4  DOUT Y#(0)=ON
```

11. Click <Command>-<2 logic>-<3 WAIT>, prompt follow window:

Inst correct

Input related parameters, press . Instruction will show in program edit window.

```
1  MOVJ VJ=50.0% PL=0
2  MOVJ VJ=50.0% PL=0
3  MOVL VL=100.0MM/S PL=0
4  DOUT Y#(0)=ON
5  WAIT X#(0)==ON DT=0 CT=0
```

12. Repeat the above steps, and input the every program points and instructions as follows:

```
1  MOVJ VJ=50.0% PL=9 TOOL=1
2  MOVJ VJ=50.0% PL=9 TOOL=1
3  MOVL VL=200.0MM/S PL=0 TOOL=1
4  DOUT Y#(0)=ON
5  WAIT X#(0)==ON T=0
6  MOVL VL=200.0MM/S PL=9 TOOL=1
7  MOVJ VJ=50.0% PL=9 TOOL=1
8  MOVJ VJ=50.0% PL=9 TOOL=1
9  MOVJ VJ=50.0% PL=9 TOOL=1
10 MOVL VL=100.0MM/S PL=0 TOOL=1
11 DOUT Y#(0)=OFF
12 WAIT X#(0)==OFF T=0
13 MOVL VL=200.0MM/S PL=9 TOOL=1
14 MOVJ VJ=50.0% PL=9 TOOL=1
15
```

Save

Close

13. Click sub menu , then click close program edit window

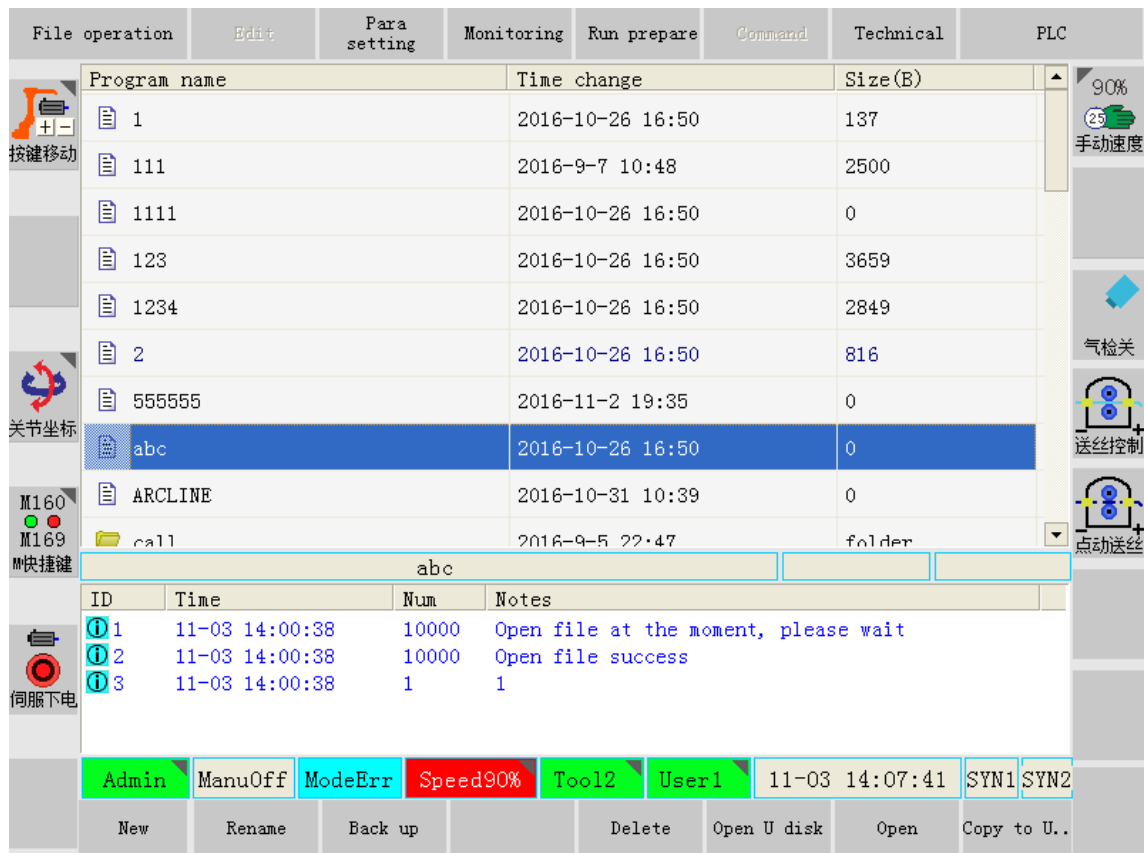
The lifting program is created by the above steps.

8 Program edit functions

Program edit functions mainly include: program list edit, mainly used to create, rename, backup, delete, and etc. of program; Program interface edit, mainly used to copy, cut, delete, find, replace and etc. of program line.

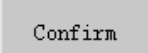
8.1 Edit in Program list

Create, rename, backup, delete, open U disk, oped or copy to U disk.

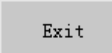


8.1.1: Create program

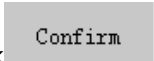
Click button, Create program window will prompt , type the

program name in the blank, click , Close input window. At this time, cursor will jump to the new program just created, and highlight it. If input the already-exist program name, system will highlight the program.

Program name can be any letters, number, Chinese character, and their combination.

If users want to cancel the input, press  to quit.

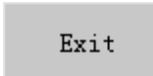
8.1.2 : Rename the already-exist program

Users should move the cursor at the program need renaming, then click  , windows as follows will prompt:  , type the modified name in the blank, and click  , the input window will close. At this time, cursor will jump at the renamed program in the program list.

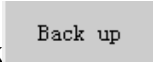
Program name can be any letters, number, Chinese character, and their combination.

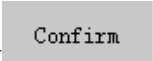
If the typed name is the same as one already-exist program , the info area will show:

1 test 更名为 test 存在相同的名称, 更名失败!

If users want to cancel the input, press  to quit.

8.1.3 : Copy the current program and paste into another program with another name.

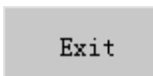
Users should move the cursor at the program need backup, then click  , windows as follows will prompt: 

Type the new name in the blank, and click  , the input window will close. At this time, cursor will jump at the renamed program in the program list.

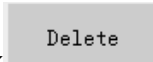
Program name can be any letters, number, Chinese character, and their combination.

If the typed name is the same as one already-exist program , the info area will show:

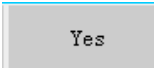
1 test 更名为 test 存在相同的名称, 更名失败!

If users want to cancel the input, press  to quit.

8.1.4 : Delete the current program

Users should move the cursor at the program need deleting, then click  , windows as follows will prompt:

Whether delete test Program?

 , confirm delete.The program will diappear.

No

If user wants to cancel press , button to quit

Description

Delete is irreversable, so be cautious with delete operation.

Open U disk

8.1.5 : Open the content list of U disk

This list only shows the user program that system can recognize; other types of document will not show. If U disk is not inserted or not recognized, system will show: please check if U disk is inserted or disinstalled. Users should insert U disk again.

Copy to
system

8.1.5.1: Copy the program to system.

Copy to
system

This button is to copy the selected program (by cursor) to the system program list. Click , system will prompt:

Whether to copy program from U disk To system?

Yes

Click , Copy the selected program to the system program list. If users want to cancel,

No

press button to quit.

Un U disk

8.1.5.2 Uninstall the current U disk:

Uninstall the current U disk, and quit the U disk content disk, return to system program list. Info bar shows: U disk uninstall succeed.

Exit

8.1.5.3 EXIT :

Quit from U disk content, and return to system program list.

Open

8.1.6 : Open the selected program and enter the program edit interface.

Description

1. Teach program and program edit should be operated in program edit interface.
2. Operation in play mode should also require program is open.
3. Operation in remote mode doesn't need open the program manually. After the power-on of driver, system will automatically open the program, whose name has been set in the remote process.

Users should move cursor at the program users want to open, and then click the button; system will enter the program edit interface of selected program. The already-exist program line will show as follows:

```
1 MOVJ VJ=10.0% PL=0
2 MOVJ VJ=10.0% PL=0
3 MOVL VL=100.0MM/S PL=0
```

Copy to U..

8.1.7 Copy the program to U disk

Insert the U disk system can recognize, then select the program (by cursor) that users want to copy. Click the button, the following window will prompt:

Sending or not fxqhtest To u disk

Yes

click

Copy the selected program and send it to U disk. If users want to cancel, press

No

button to quit. After copy, click

Open U disk

to check if there is the program in the U disk

content list. Check if the copy has completed.

8.2 Edit in program edit interface

copy-copy current line、copy-copy section、paste、cut-cut current line、cut-cut section、delete、find、replace、turn into、modify as time sequence、reset.

8.2.1 1copy->1 copy current line:

Copy cursor located lineto backstage

description
When copy the program line, teach point data will also be copied. So please pay attention to the execution position of the copied program line.

Move the cursor to the line need copying, select <copy current line>; move cursor to the position of paste, use <paste> to past the line before the cursur line. E.g., copy the line 4 ARCSTART# (0) to the line before line 8. The specific operation is following:

```

1  MOVJ VJ=50.0% PL=9 TOOL=1
2  MOVJ VJ=50.0% PL=9 TOOL=1
3  MOVJ VJ=25.0% PL=0 TOOL=1
4  ARCSTART#(0)
5  MOVL VL=50.0MM/S PL=9 TOOL=1
6  ARCEND#(0)
7  MOVJ VJ=25.0% PL=0 TOOL=1
8  MOVJ VJ=50.0% PL=9 TOOL=1
9

```

1. Move cursor at the line 4, as above figure.
2. Choose<Edit>-<1 copy>-<1 copy current line>-<confirm>, info bar shows multiple program line copy succeed.
3. Move the cursor at the line 8, as below figure.

```

1  MOVJ VJ=50.0% PL=9 TOOL=1
2  MOVJ VJ=50.0% PL=9 TOOL=1
3  MOVJ VJ=25.0% PL=0 TOOL=1
4  ARCSTART#(0)
5  MOVL VL=50.0MM/S PL=9 TOOL=1
6  ARCEND#(0)
7  MOVJ VJ=25.0% PL=0 TOOL=1
8  ARCSTART#(0)
9  MOVJ VJ=50.0% PL=9 TOOL=1
10

```

4. Choose <Edit>-<2 paste>-<confirm>, info bar shows paste succeed. The pasted line will appear at the position between line 7 and line 8, while cursor is at this line, as below figure:

```

1  MOVJ VJ=50.0% PL=9 TOOL=1
2  MOVJ VJ=50.0% PL=9 TOOL=1
3  MOVJ VJ=25.0% PL=0 TOOL=1
4  ARCSTART#(0)
5  MOVL VL=50.0MM/S PL=9 TOOL=1
6  ARCEND#(0)
7  MOVJ VJ=25.0% PL=0 TOOL=1
8  ARCSTART#(0)
9  MOVJ VJ=50.0% PL=9 TOOL=1
10

```

8.2.2 1copy->2 copy section :

Copy the contains between starting line and end line to the backstage

description
When copy the program section, teach point data will also be copied. So please pay attention to the execution position of the copied program line.

Copy the contents between starting line and end line to the backstage, and then move the cursor the position of paste, use <paste> to past the section before the cursur line. E.g., copy the line 2 and line 3 to the line before line 8. The specific operation is following:

```
1  MOVJ VJ=50.0% PL=9 TOOL=1
2  MOVJ VJ=50.0% PL=9 TOOL=1
3  MOVJ VJ=25.0% PL=0 TOOL=1
4  ARCSTART#(0)
5  MOVL VL=50.0MM/S PL=9 TOOL=1
6  ARCELD#(0)
7  MOVJ VJ=25.0% PL=0 TOOL=1
8  MOVJ VJ=50.0% PL=9 TOOL=1
9
```

1. Choose<Edit>-<1 copy>-<2 copy section>-<confirm>. The following window will prompt:

Copy first Copy end
caution: when operating block, line number displaying is

Confirm

Input 2 in the starting line, and input 3 at the end line, click

Cancel

program lines copy succeed. If users want to cancel the copy operation, click button to quit.

2. Move cursor to line 8, as below figure:

```
1  MOVJ VJ=50.0% PL=9 TOOL=1
2  MOVJ VJ=50.0% PL=9 TOOL=1
3  MOVJ VJ=25.0% PL=0 TOOL=1
4  ARCSTART#(0)
5  MOVL VL=50.0MM/S PL=9 TOOL=1
6  ARCELD#(0)
7  MOVJ VJ=25.0% PL=0 TOOL=1
8  MOVJ VJ=50.0% PL=9 TOOL=1
9
```

3. Choose<Edit>-<2 paste>-<confirm>, info bar shows past succeed. And the copied section (line 2 and line 3) will appear between the line 7 and line 8, while cursor is at the line 8, as below figure shows:

```
1  MOVJ VJ=50.0% PL=9 TOOL=1
2  MOVJ VJ=50.0% PL=9 TOOL=1
3  MOVJ VJ=25.0% PL=0 TOOL=1
4  ARCSTART#(0)
5  MOVL VL=50.0MM/S PL=9 TOOL=1
6  ARCELD#(0)
7  MOVJ VJ=25.0% PL=0 TOOL=1
8  MOVJ VJ=50.0% PL=9 TOOL=1
9  MOVJ VJ=25.0% PL=0 TOOL=1
10 MOVJ VJ=50.0% PL=9 TOOL=1
11
```

8.2.3 2paste : used to past after copy and cut

Paste the copied or cut program into the line where cursor is located.

8.2.4 3cut->1 cut the current line :

Copy the line where cursor is and delete this line.

Move the cursor to the line need cutting, then click “cut current line”. The selected line will be copied to backstage and current line will be deleted. Move cursor to the desired position, click <paste>, and the cut line will appear at the cursor position. E.g. cut the line 4 ARCSTART# (0) to the position before line 8. The specific steps are as follows:

```
1  MOVJ VJ=50.0% PL=9 TOOL=1
2  MOVJ VJ=50.0% PL=9 TOOL=1
3  MOVJ VJ=25.0% PL=0 TOOL=1
4  ARCSTART#(0)
5  MOVL VL=50.0MM/S PL=9 TOOL=1
6  ARCELD#(0)
7  MOVJ VJ=25.0% PL=0 TOOL=1
8  MOVJ VJ=50.0% PL=9 TOOL=1
9
```

1. Move cursor to the line 4, as above figure.

2. Click<Edit>-<3 cut>-<1 cut current line>-<confirm>. System will show: current line has benn cut successfully. At this time, line 4 ARCSTART# (0) is copied to the backstage. And line 4 is deleted in program edit interface, as below figure:

```
1  MOVJ VJ=50.0% PL=9 TOOL=1
2  MOVJ VJ=50.0% PL=9 TOOL=1
3  MOVJ VJ=25.0% PL=0 TOOL=1
4  ☐MOVL VL=50.0MM/S PL=9 TOOL=1
5  ARCEND#(0)
6  MOVJ VJ=25.0% PL=0 TOOL=1
7  MOVJ VJ=50.0% PL=9 TOOL=1
8
```

Cancel

Confirm

If users want to cancel the cut operation, press to quit before clicking . If

users want to restore the cut line, move cursor to the proper position and paste the cut line back.

3. Move the cursor to the line 8 (line 7 after cut).

```
1  MOVJ VJ=50.0% PL=9 TOOL=1
2  MOVJ VJ=50.0% PL=9 TOOL=1
3  MOVJ VJ=25.0% PL=0 TOOL=1
4  MOVL VL=50.0MM/S PL=9 TOOL=1
5  ARCEND#(0)
6  MOVJ VJ=25.0% PL=0 TOOL=1
7  ☐MOVJ VJ=50.0% PL=9 TOOL=1
8
```

Click<Edit>-<2 paste>-<confirm>. At this time, the cut line will appear at the position above line 8, as below figure:

```
1  MOVJ VJ=50.0% PL=9 TOOL=1
2  MOVJ VJ=50.0% PL=9 TOOL=1
3  MOVJ VJ=25.0% PL=0 TOOL=1
4  MOVL VL=50.0MM/S PL=9 TOOL=1
5  ARCEND#(0)
6  MOVJ VJ=25.0% PL=0 TOOL=1
7  ☐ARCSTART#(0)
8  MOVJ VJ=50.0% PL=9 TOOL=1
9
```

8.2.5 3cut->2 cut section:

Copy the contains between starting line and end line to the backstage, and delete them

Cut the content between starting line and end line to the backstage, and then delete them. Move the cursor the position of paste, use <paste> to paste the section before the cursor line. E.g., copy the line 4, 5, and 6 to the line before line 8. The specific operation is following:

```
1  MOVJ VJ=50.0% PL=9 TOOL=1
2  MOVJ VJ=50.0% PL=9 TOOL=1
3  MOVJ VJ=25.0% PL=0 TOOL=1
4  ☐ARCSTART#(0)
5  MOVL VL=50.0MM/S PL=9 TOOL=1
6  ARCEND#(0)
7  MOVJ VJ=25.0% PL=0 TOOL=1
8  MOVJ VJ=50.0% PL=9 TOOL=1
9
```

Choose<Edit>-<3 shear>-<2 cut block>. The following window will prompt:

Cut first Cut end
caution: when operating block, line number displaying is

Input 4 in

Confirm

the starting line. Input 6 at the end line, click , and info bar shows multiple program lines cut succeed. At this time, the content between line 4 and line 6 has been deleted:

```
1  MOVJ VJ=50.0% PL=0
2  MOVJ VJ=50.0% PL=0
3  MOVJ VJ=50.0% PL=0
7  MOVJ VJ=50.0% PL=0
8  MOVJ VJ=50.0% PL=0
```

Cancel

If users want to cancel the cut operation, click **Cancel** button to quit. If users want to restore the cut lines, move cursor to the proper position and paste the cut lines back.

2. Move cursor to line 8.

3. Choose<Edit>-<2 paste>, info bar shows paste succeed. And the cut section (line 4, 5, 6) will appear between the line 7 and line 8, while cursor is at the line 8, as below figure shows:

```
1  MOVJ VJ=50.0% PL=9 TOOL=1
2  MOVJ VJ=50.0% PL=9 TOOL=1
3  MOVJ VJ=25.0% PL=0 TOOL=1
4  ARCSTART#(0)
5  MOVL VL=50.0MM/S PL=9 TOOL=1
6  ARCEMD#(0)
7  MOVJ VJ=25.0% PL=0 TOOL=1
8  MOVJ VJ=50.0% PL=9 TOOL=1
9
```

8.2.6 4Delete:

Delete the line where cursor is located

Description
Delete of program line is irreversable, please be cautious with delete operation!

Move cursor at the line users want to delete. Click <Edit>-<4 delete>-<confirm>. Info bar will show:

Whether delete current line program

If users want to cancel the delete operation, press cancel button to quit before pressing confirm.

Notice: After pressing confirm button, the deleted line cannot be restored.

8.2.7 5find:

To find specific content, cursor will locate at the found line.

Firstly, move the cursor at the first line, click <edit>-<5 Search>, and following window will prompt:

Search

Then select the addition users want to search, and input the addition values, as shown in above.

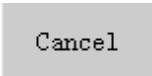
5 Search

Then click 5 Search, system will search. The nearest line with the same addition value will be found, and cursor will locate at the line. System prompt: find. If no addition value has been found, system will prompt: no result.

If there are many addition value is the same with the search condition, every time user clicks

5 Search

button, system will search from the cursor position and find the nearest addition value. The rest can be found in the same manner.

If users don't want to continue, click  to quit.

Additions users can search include: PL、VJ、VL、X、Y、M、GP、LP、GD、LD、GI、LI。

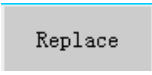
8.2.8 6Replace :

Replace the original content with the new content.

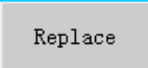
Firstly, move the cursor at the first line, click <Edit>-<6 replace> , and following window will prompt:

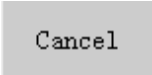


Then select the addition users want to replace, and input the original addition values 0 and replaced

value 9, as shown in above. Then click , system will search. The nearest line with the same addition value will be found, original value will be replaced by new one and cursor will locate at the line. System prompt: replace succeed. If no addition value has been found, system will prompt: no result.

If there are many addition value is the same with the search condition, every time user clicks

 button, system will search from the cursor position and find the nearest addition value. The rest can be found in the same manner, until every additions with the original value have been replaced.

If users don't want to continue, click  to quit.

Additions users can search include: PL、VJ、VL、X、Y、M、GP、LP、GD、LD、GI、LI。

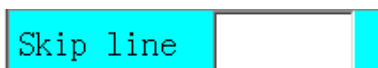
8.2.9 7 Turn:

Locate the cursor at specific line.

E.g.: move cursor to line 6 using Turn function.

```
1  MOVJ VJ=50.0% PL=9 TOOL=1
2  MOVJ VJ=50.0% PL=9 TOOL=1
3  MOVJ VJ=25.0% PL=0 TOOL=1
4  ARCSTART#(0)
5  MOVL VL=50.0MM/S PL=9 TOOL=1
6  ARCEMD#(0)
7  MOVJ VJ=25.0% PL=0 TOOL=1
8  MOVJ VJ=50.0% PL=9 TOOL=1
9
```

Click <Edit>-<7 Go to>, following window will prompt.



Type the number 6 into the blank. And click <  > button. Then cursor will be located at the line 6, as below figure.

```
6  MOVJ VJ=50.0% PL=0
7  MOVJ VJ=50.0% PL=0
```

If users don't want to move the cursor at the turn window, users can press  button to quit.

8.2.10 8Modify as timesequence :

Backup functions, under development, not available now

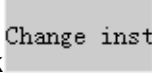
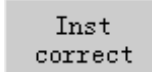
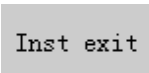
8.2.11 9 Program reset :

Reset to original condition

Click <Edit>-<9 turn>-<confirm>, system will prompt: Reset.

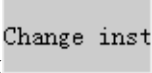
8.2.12 Modify instructions:

Modify the current program line.

Move cursor to the line users want to modify, click , windows prompt the current program edit interface. After the modification, users can click  to confirm modification, or click  to cancel the modification.

The specific steps are as follows:

```
1  MOVJ VJ=50.0% PL=9 TOOL=1
2  MOVJ VJ=50.0% PL=9 TOOL=1
3  MOVJ VT=25.0% PL=0 TOOL=1
```

Move cursor at the line 2, click , following window will prompt:



In this status, click submenu <move> or <logic>, or click main menu< Command > to select other instructions to modify the current line. It's also allowed to modify the addition without program modification.

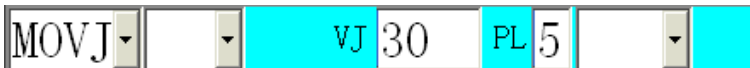
Description

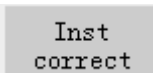
As for modification of the move instructions with move gesture, it's necessary to hold the security switch to record the position and gesture.

As for modification of the move instructions without move gesture, it's unnecessary to hold the security switch.

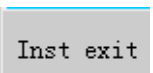
As for modification of the non-move instructions, it's unnecessary to hold the security switch.

Modify the VJ speed to 30%, PL to 5, as following:



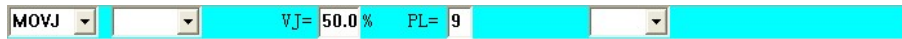
Click  confirm modification. Then program is:

```
6  MOVJ VJ=30.0% PL=5
7  MOVJ VJ=50.0% PL=0
```

Click  button to cancel modification.

8.2.13 move: Call move instruction

In program edit interface, click this button, system will prompt the move instructions windows as follows (can also be called by < Command >-<Movement>-<MOVJ>):



Click this button for many times, the instructions will change in the order of the list < Command >-<Movement>, e.g.MOVJ-MOVL-MOVC-MOVJ.....

8.2.14 logic: Call logic instructions

In program edit interface, click this button, system will prompt the logic instructions windows as follows (can also be called by <Movement>-<1 logic>-<1 DOUT>-<confirm>):



Click this button for many times, the instructions will change in the order of the list < Command >-<logic >-<confirm>, e.g. DOUT-AOUT-WAIT-TIME-PAUSE-JUMP.....

8.2.15 Open process: Open process button

When program line uses CALL instruction, and program calls the process document, users can open the process interface by pressing this button.

8.2.16 last instruction: Call last instruction

If other instructions have been used before, the last instruction window will be open after pressing this button.

e.g. In program edit interface, MOVJ line is created as follows:



Click <previous instruction>, the MOVJ window used last time will appear:



This button is mainly for convenient call of the same instructions.

8.2.17 save: Save the program

After the program editing, click the save button to save the program. Then status bar will show document saved.

8.2.18 close: Close the program edit interface.

Clicking this button can close the program edit window. However, if program hasn't been saved, the edited content may lost.

9 Program test-run

Program test-run refer to: after the editing of program, users can allow robot to operate line by line of the program by certain operation. This is useful to help user to compare the actual move and move track and to judge is move or track is correct.

9.1 Preparation:

9.1.1 Related parameters

parameter type	Parameter	value	description
Operation parameters (integrator authority)	Cursor move in order in test-run	0	no move: Cursor stop at the line after test-run
		1	move downwards: Cursor stop to the next line after test-run

Specific steps for parameters modification:

Firstly, <parameter settings>-<7 system parameters>-<1 operation authority selection>-<confirm>. Input the integrator password in the prompt interface, click <confirm>, the authority will be changed to integrator authority.

Then, < parameter settings >-<3 operation parameters>-<confirm>. Choose [cursor move in test-run] using cursor in the prompt program list. Click <modify> in the sub menu. Input number 0 or 1 in the prompt input window, click <confirm> to finish the modification of parameter of cursor move in test-run.

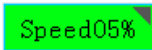
9.1.2 Other preparation

1 Switch to robot movement allowance status  按键移动.

2 Modify the speed into a proper speed by pressing  or  button or speed multiplier



It's suggested that speed multiplier is less than 50%. The modified speed multiplier can be found

in status display area .

9.2 Steps for program test-run:

1. Return to program list interface as follows:

Program name	Time change	Size(B)	
1	2016-10-26 16:50	137	
111	2016-9-7 10:48	2500	
1111	2016-10-26 16:50	0	
123	2016-10-26 16:50	3659	
1234	2016-10-26 16:50	2849	
2	2016-10-26 16:50	816	
555555	2016-11-2 19:35	0	
abc	2016-10-26 16:50	0	
ARCLINE	2016-10-31 10:39	0	
call	2016-9-5 22:47	folder	

2. Move cursor to the line users want to run, as the 555555 in above figure. Then click submenu <open> button to open this program. Users will enter the program edit interface, as follows:

```

1  MOVJ VJ=50.0% PL=0
2  MOVJ VJ=50.0% PL=0
3  MOVJ VJ=50.0% PL=0
7  MOVJ VJ=50.0% PL=0
8  MOVJ VJ=50.0% PL=0

```

3. Move cursor at the beginning of the line that users want to test-run, for example, at the beginning of second line.



4. Holding security switch, then press  button in character keyboard. System will control the robot to execute the current line, such as robot movement, IO output, calculation, logic.....

Notice

When cursor is located within the IF, WHILE, SWITCH structure, system will prompt error alert. So before the test-run, move the cursor beyond the above instruction structure.

When MOVJ is test-run of define the points, robot move track will be straight.